Agentic AI on AWS, hands-on labs by Shivank Agrawal

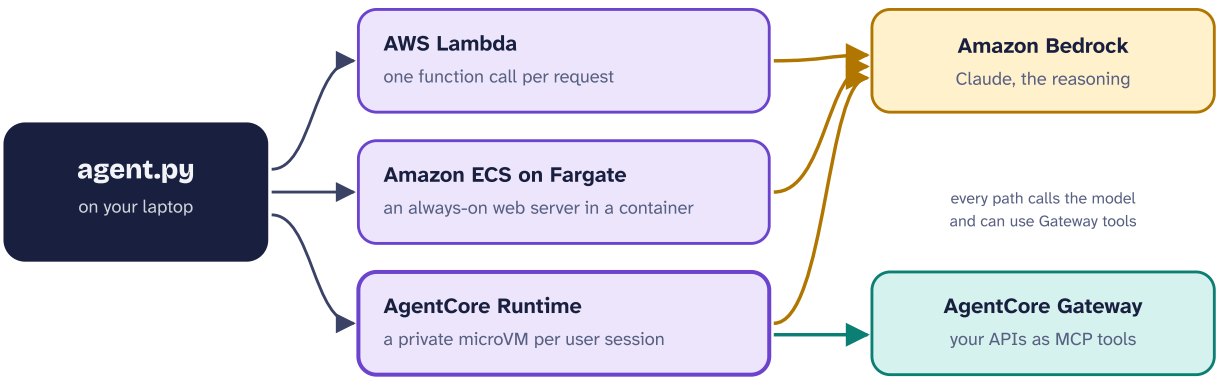
# Shipping agents to production

Section 6. Everything so far has run on your laptop. Now you'll put agents behind real URLs on AWS three ways (a **Lambda** function, an always-on **ECS** container, and **AgentCore Runtime**), turn an existing REST API into agent tools with **AgentCore Gateway**, and learn the patterns for plugging agents into enterprise systems.

Folder `section-06-production-deployment`

Labs `lambda` `ecs-fargate` `agentcore-runtime` `agentcore-gateway` `multi-agent-on-agentcore`

Model: Claude Sonnet 4.6 on Amazon Bedrock




**One agent, three homes.** The agent code barely changes; what changes is how it's woken up, how long it can run, what it remembers between requests, and how much of the plumbing you build yourself.

# 6.0 From laptop to production

Over the last five sections you built agents that reason, use tools, remember conversations, work in teams and talk over MCP and A2A. All of it runs nicely on your laptop. Production is a different place.

The moment someone says *"let's put this in front of real users"*, questions appear that nobody asked during development. What happens when 200 people hit your agent at the same time? What happens when the model provider has a bad day? If the agent crashes at 3 AM and nobody notices until Monday, what then?

The AI part is rarely what slows teams down. It's the infrastructure: networking, IAM (Identity and Access Management) policies, containers, load balancers. A two-day deployment task becomes a week of troubleshooting because nobody on the team has deployed a containerised application before, and the error messages are hard to decode.

This section closes that gap. It won't turn you into a DevOps engineer, but it gives you enough to get agents running in production. For hardened deployments with custom networking, compliance requirements or multi-region failover, you'll still want to partner with your infrastructure team.

> ✦ **Think of it like this**
>
> Cooking a great dish for your family is development. Opening a restaurant is production. The recipe is the same, but now you need a kitchen that's open on time every day, enough staff for the Friday rush, a fire extinguisher, a health inspection, a way to know when a dish went wrong, and a budget that doesn't burn the place down. This section is about the restaurant, not the recipe.

## What you'll learn here

1. What actually changes when you move from development to production, and a readiness checklist
2. The four ways to run agents on AWS and how to choose
3. Deploying on AWS Lambda (serverless, event-driven) and on Amazon ECS with Fargate (always-on containers), with a lab for each
4. Amazon Bedrock AgentCore: its architecture, Runtime, Gateway and Identity, with three labs
5. Enterprise integration patterns for databases, APIs and events, and four industry use cases

## Before you start

**1** **A working Strands setup from earlier sections**
Plus basic Python functions and classes.

**2** **An AWS account with permissions to create resources**
The labs create IAM roles, container registries, Lambda functions, ECS services, load balancers and AgentCore resources, and call Claude on Amazon Bedrock. Set `AWS_REGION=us-east-1` (or your Bedrock region).

**3** **Docker, for the Lambda and ECS labs**
Familiarity with Docker helps but isn't required; this page explains what you need. The AgentCore labs don't need Docker on your machine.

**4** **Command-line tools**
AWS CLI v2 for everything; the AWS SAM CLI for the Lambda lab; `pip install bedrock-agentcore-starter-toolkit` for the AgentCore labs; `jq` for the multi-agent lab. On Windows, run the shell scripts from WSL (recommended) or Git Bash.

**5** **An IDE (optional)**

VS Code, Kiro, Claude Code or Cursor, and Jupyter for the two notebooks.

| Folder | Target | Example | On this page |
|---|---|---|---|
| `lambda/` | AWS Lambda behind API Gateway | Serverless contract-analysis agent | 6.3 |
| `ecs-fargate/` | Amazon ECS on Fargate behind a load balancer | Hospital surgical-scheduling agent with a chat UI | 6.4 |
| `agentcore-runtime/` | AgentCore Runtime | Contract-analysis agent deployed with the starter toolkit | 6.6 |
| `agentcore-gateway/` | AgentCore Gateway | A REST API (OpenAPI spec in S3) turned into MCP tools | 6.8 |
| `multi-agent-on-agentcore/` | AgentCore Runtime (MCP + A2A) | Finance orchestrator → stock agent (A2A) → stock tools (MCP) | 6.10 |
| `setup_cognito.sh` | Amazon Cognito | A user pool and OAuth token for the multi-agent deployment | 6.10 |

terminal · quick start Copy

```
# AgentCore Runtime
cd agentcore-runtime && jupyter lab deploy_to_agentcore_runtime.ipynb

# ECS Fargate
cd ecs-fargate && chmod +x deploy.sh cleanup.sh && ./deploy.sh      #

# Lambda
cd lambda && chmod +x deploy.sh cleanup.sh && ./deploy.sh           #
```

```
# AgentCore Gateway
cd agentcore-gateway && jupyter lab gateway_openapi_target.ipynb

# Multi-agent (local first, then AgentCore) — see multi-agent-on-agent
```

> ! **These labs cost real money until you clean up**
>
> Unlike earlier sections, these labs create AWS resources that keep running (and billing) after your script finishes: a load balancer bills by the hour whether anyone uses it or not. Every lab ends with a clean-up step. Run it the same day.

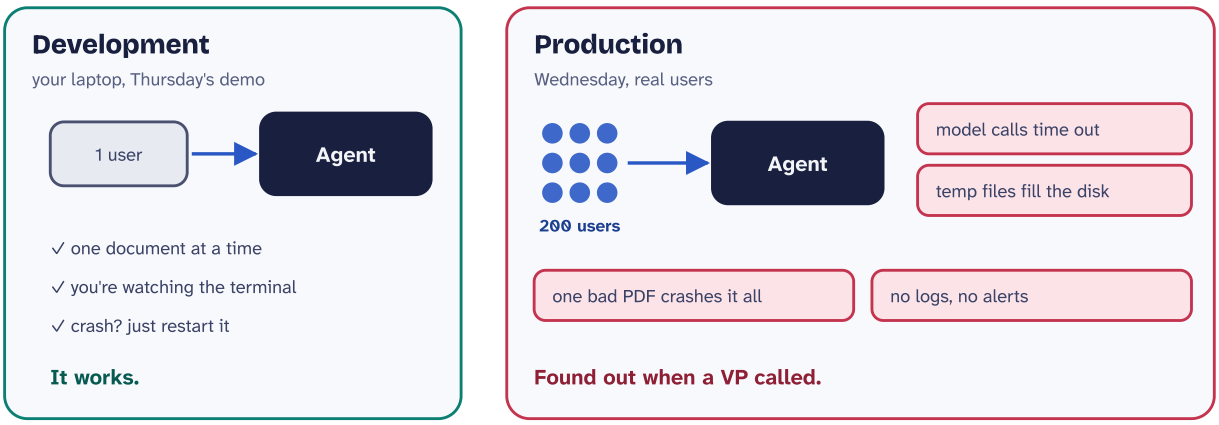
> i **One model setting for every lab**
>
> All agents default to `global.anthropic.claude-sonnet-4-6`. Set `BEDROCK_MODEL_ID` before deploying to change it; the deploy scripts and notebooks pass it through to the deployed agent.

---

## 6.1 What actually changes in production

A true-to-life story first. A team built a brilliant document analysis agent. It parsed PDFs, extracted key terms and summarised findings flawlessly in development. They demoed it to leadership on Thursday, got approval on Monday, and were in trouble by Wednesday.

The agent worked great for one document at a time. In production, 200 users hit it at once. Model calls started timing out. Temporary files piled up and filled the disk. One malformed PDF crashed the whole process, and there was no recovery: just silence. No logs, no alerts, nothing. They found out it was down when a VP called to complain.




**Same code, different world.** Nothing in the agent's reasoning changed. What broke was everything around it: concurrency, resources, failure handling and visibility.

This is a normal trajectory when you skip the production-readiness conversation. Here's what changes:




**Five habits that don't survive production.** Each one has a production replacement; this section and the next show you how to put most of them in place.

## Reliability isn't optional

You need error handling, retries, circuit breakers and graceful degradation, so one failure doesn't take everything down.

### Security becomes real

Proper secrets management, least-privilege IAM roles and network isolation replace the admin keys on your laptop.

### Debugging gets harder

With many containers and interleaved requests you need structured logs, distributed tracing and observability. Section 7 covers this in depth; this section lays the groundwork.

## Are you ready? Seven questions

Teams that rush to deploy often spend twice as long fixing things they should have handled up front. Before you go to production, answer these honestly. If most are missing, that's fine: this section and the next cover them.

Try it **Production readiness check**

- [ ] Error handling that goes beyond try/except: pass
- [ ] You can tell, right now, what your agent did for its last 100 requests
- [ ] If the model provider has an outage, the agent degrades gracefully instead of crashing or hanging
- [ ] Resource limits (CPU, memory, IO), so a runaway agent can't consume everything
- [ ] Someone besides you has run this code successfully
- [ ] You can roll back if a new version breaks something
- [ ] You've estimated what it will cost per month

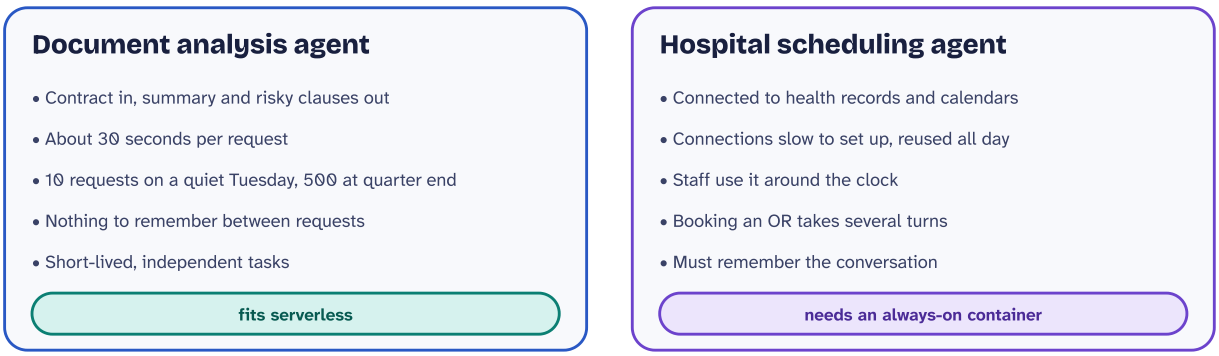
READY
**0** of 7

READINESS

Tick the ones you can honestly answer "yes" to.

## 6.2 Choosing a deployment path

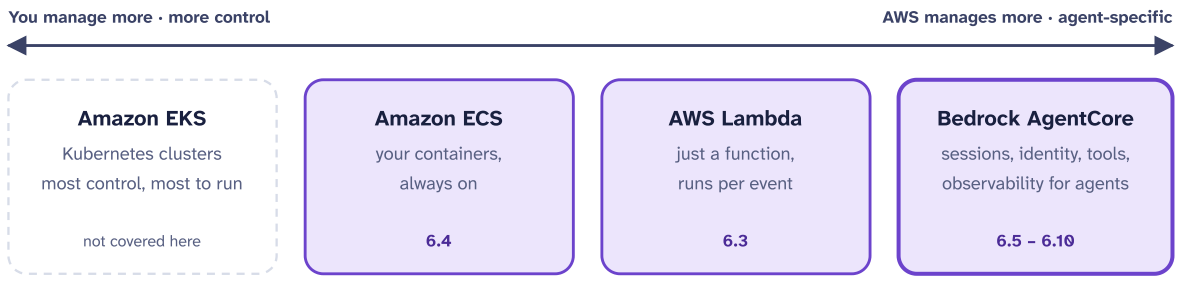
Different agents need different homes. Here are two that look similar on paper and need completely different infrastructure.




**Same kind of code. Completely different infrastructure needs.**

**The workload picks the platform.** Short, independent and spiky points one way; long-lived, stateful and always-on points the other.

AWS gives you four options for running agents in production: **AWS Lambda**, **Amazon Elastic Container Service (ECS)**, **Amazon Elastic Kubernetes Service (EKS)** and **Amazon Bedrock AgentCore**. Each fits a different set of constraints. Picking the wrong one won't end your project, but it can cost you weeks of rework.




**A spectrum, not a ladder.** More control means more to build and run yourself. None is "better"; each fits a shape of workload.

✦ **Think of it like this**

**Lambda is a taxi:** you pay per ride, nothing is waiting between rides, and the driver forgets you when you get out. **ECS is a company car:** always parked outside and ready, but you pay for it all day and you maintain it. **AgentCore Runtime is a hotel room:** each guest gets a private room for their stay, housekeeping is included, and the room is wiped clean at checkout.

| | Lambda | ECS Fargate | AgentCore Runtime |
|---|---|---|---|
| **Shape** | Function per request | Always-on web service | Managed agent runtime, one microVM per session |
| **Max run time** | 15 min (API Gateway: 29 s by default) | Unlimited | Up to 8 hours per session |
| **You manage** | Packaging | Containers, load balancer, scaling | Just the agent code |
| **Good for** | Short, bursty, event-driven tasks | Existing container platforms, custom web apps | Long-running, stateful, multi-user agents |

Let's explore each path, starting with the simplest.

---

## 6.3

# AWS Lambda: serverless and event-driven

Back to the document analysis agent. A customer uploads a contract and the agent needs to process it. You don't want a server running 24/7 waiting for

uploads that might come once an hour or once a minute. You want your code to run when a document arrives, process it, and stop.

AWS Lambda lets you attach code directly to an event. You write a **handler** function, connect it to a **trigger** (an S3 upload, an API Gateway request, a scheduled event), and Lambda runs it when the trigger fires. Nothing runs, and nothing is billed, in between. That's what serverless means: there are still servers, you just never see or manage them.




**Wake up, do the job, go back to sleep.** The handler is the only entry point; triggers decide when it runs.

## The handler, from the course code

lambda/src/app.py · setup that runs once per environment Copy

```
import json
import os

from strands import Agent
from strands.models.bedrock import BedrockModel

MODEL_ID = os.environ.get("MODEL_ID") or os.environ.get("BEDROCK_MODEL

SYSTEM_PROMPT = """You are a document analysis assistant.
When given a contract or document, you:
1. Extract key terms and parties involved
2. Flag any risky or unusual clauses
3. Provide a clear summary
```

```
Be concise and specific. Cite the exact language from the document whe

# Initialize the model outside the handler so it is reused across warm
# The first request pays the setup cost; later requests on the same co
model = BedrockModel(model_id=MODEL_ID)
```

Notice where `model` is created: **outside** the handler, at module level. Lambda doesn't start a fresh Python process for every request. While traffic is flowing it reuses the same execution environment, and module-level objects survive between calls. The first request pays the setup cost (importing Strands, creating the Bedrock client); every later request on that environment skips straight to the handler. Move that setup inside the handler and you pay it on every single call.

**lambda/src/app.py · the handler, runs on every request** Copy

```
def lambda_handler(event, context):
    """Handle API Gateway POST requests."""
    # Parse the request body
    if isinstance(event.get("body"), str):
        body = json.loads(event["body"])
    else:
        body = event.get("body") or event

    document_text = body.get("document_text", "")

    if not document_text:
        return {
            "statusCode": 400,
            "headers": {"Content-Type": "application/json"},
            "body": json.dumps({"error": "Missing 'document_text' in r
        }

    # A fresh Agent per request: a warm container serves many differen
    # so conversation history must never carry over from one request t
    agent = Agent(model=model, system_prompt=SYSTEM_PROMPT, callback_h
    result = agent(f"Analyze this contract:\n\n{document_text}")

    return {
        "statusCode": 200,
        "headers": {"Content-Type": "application/json"},
```

```
        "body": json.dumps({"analysis": str(result)}),
    }
```

**1** **Parse the body**

API Gateway delivers the HTTP body as a JSON *string*; a direct test invocation might pass a dict. The code handles both.

**2** **Validate input**

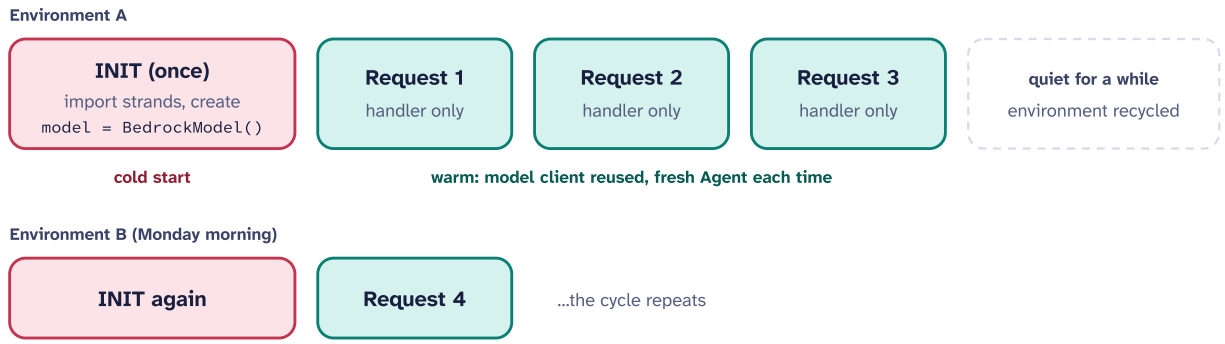
No `document_text`? Return a `400` with a clear error instead of sending an empty prompt to the model.

**3** **A fresh `Agent` per request**

The model client is shared, but the agent is not. A warm environment serves many different callers, and an agent keeps its conversation in `agent.messages`. Reuse one agent and customer B's request would carry customer A's contract in its history. Expensive and stateless: share it. Cheap and stateful: create it per request.

**4** **Return an HTTP response**

A `statusCode`, headers and a JSON body, which API Gateway turns into the HTTP reply.




**Pay the setup once per environment, not once per request.** Anything at module level runs in INIT; anything inside the handler runs on every call.

Try it **Cold starts, warm starts and where your setup code lives**

☐ Provisioned concurrency: keep 2 environments pre-warmed

Execution environments

no environments
nothing running, nothing billed

| REQUESTS | COLD STARTS | LAST SETUP DELAY | IDLE ENVIRONMENTS YOU PAY FOR |
|---|---|---|---|
| 0 | 0 | – | 0 |

Send a request to see what happens. Setup delays here are illustrative (cold starts are typically a few seconds).

## Packaging and deploying with SAM

The deployment uses an AWS SAM (Serverless Application Model) template: a YAML file where you define the Lambda function, the API Gateway trigger, the IAM permissions and the container configuration, then deploy it all with one command.

SAM turns it into a CloudFormation stack, so everything is created, and later deleted, together.

**lambda/template.yaml · settings and parameters** Copy

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: Strands document analysis agent on AWS Lambda behind API

Globals:
  Function:
    Timeout: 900
    MemorySize: 1024

Parameters:
  Architecture:
    Type: String
    Default: arm64
    AllowedValues:
      - arm64
      - x86_64
    Description: "Lambda architecture. Use arm64 for Apple Silicon / G
  ModelId:
    Type: String
    Default: global.anthropic.claude-sonnet-4-6
    Description: "Bedrock model or inference profile ID used by the ag
```

`Timeout: 900` is Lambda's maximum (15 minutes) and `MemorySize: 1024` gives the function 1 GB (Lambda also scales CPU with memory). The two parameters let the deploy script choose the CPU architecture and the model without editing the template.

**lambda/template.yaml · the function, its trigger, a bucket and the endpoint output** Copy

```
Resources:
  DocumentAnalysisFunction:
    Type: AWS::Serverless::Function
    Properties:
      PackageType: Image
      Architectures:
        - !Ref Architecture
      Environment:
```

```
          Variables:
            MODEL_ID: !Ref ModelId
        Policies:
          - Statement:
              - Effect: Allow
                Action:
                  - bedrock:InvokeModel
                  - bedrock:InvokeModelWithResponseStream
                Resource:
                  - "arn:aws:bedrock:*::foundation-model/*"
                  - "arn:aws:bedrock:::foundation-model/*"
                  - !Sub "arn:aws:bedrock:*:${AWS::AccountId}:inference-
          - S3ReadPolicy:
              BucketName: !Ref DocumentBucket
        Events:
          AnalyzeContract:
            Type: Api
            Properties:
              Path: /analyze
              Method: post
      Metadata:
        DockerTag: v1
        DockerContext: ./src
        Dockerfile: Dockerfile

    DocumentBucket:
      Type: AWS::S3::Bucket

  Outputs:
    ApiEndpoint:
      Description: API Gateway endpoint URL
      Value: !Sub "https://${ServerlessRestApi}.execute-api.${AWS::Regio
```

Two pieces are easy to get wrong and hard to debug when you do.

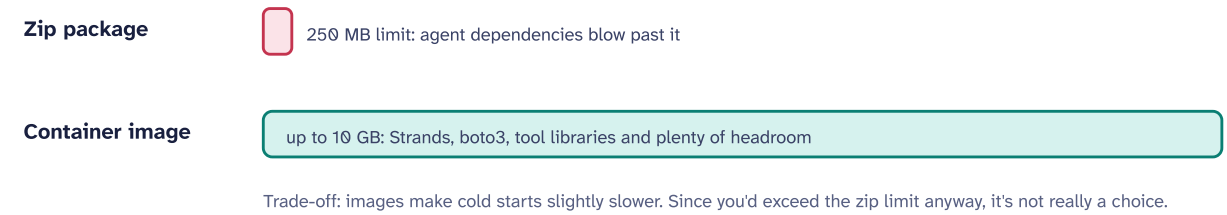
## 1. The IAM policy needs more than one resource

| Resource ARN | Why it's there |
| --- | --- |
| `arn:aws:bedrock:*::foundation-model/*` | Calling a model directly, in any region |

| Resource ARN | Why it's there |
| --- | --- |
| `arn:aws:bedrock:::foundation-model/*` | The region-less form used behind `global.` profiles, which can route a request to any region |
| `arn:aws:bedrock:*:${AWS::AccountId}:inference-profile/*` | The inference profile itself. IDs that start with `us.` or `global.` (like `global.anthropic.claude-sonnet-4-6`) are inference profiles, not models. |

> ! **The error won't tell you what's missing**
>
> Leave out the inference-profile resource and you get an `AccessDeniedException` that doesn't clearly say which permission is missing. Include all of them from the start. Each ARN (Amazon Resource Name) is just AWS's unique address for a resource.

## 2. Ship a container image, not a zip




`PackageType: Image` tells SAM to build a Docker image from `src/Dockerfile` and Lambda runs it.

`lambda/src/Dockerfile` Copy

```
FROM public.ecr.aws/lambda/python:3.13

COPY requirements.txt ${LAMBDA_TASK_ROOT}
RUN pip install -r requirements.txt

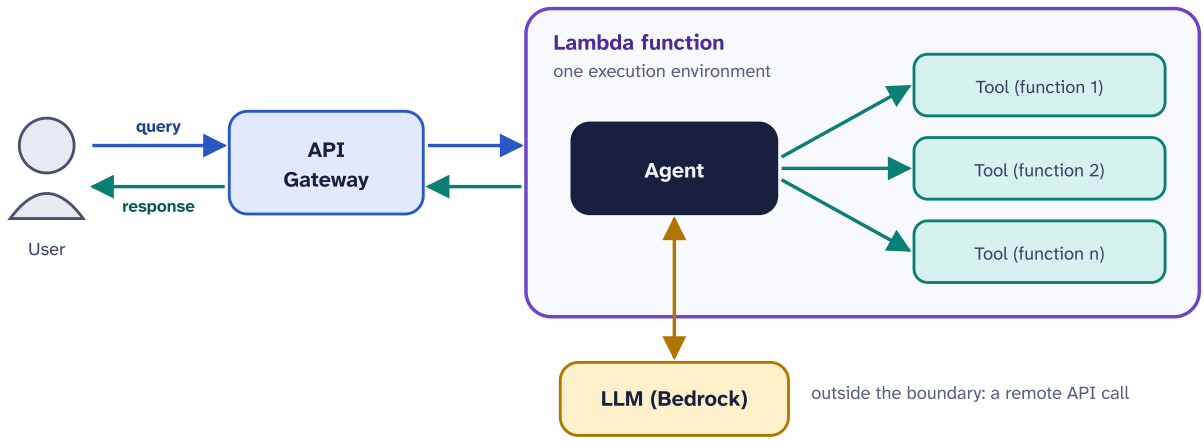
COPY app.py ${LAMBDA_TASK_ROOT}
```

```
CMD ["app.lambda_handler"]
```

The base image is AWS's official Lambda Python image. `CMD ["app.lambda_handler"]` tells Lambda which function in which file is the handler. The model ID lives in an environment variable ( `MODEL_ID` ), so you can switch models by redeploying with a different parameter, without touching the code.

> **Model IDs must be exact**
>
> Copy model IDs exactly as Bedrock lists them. Some include a version suffix ( `global.anthropic.claude-haiku-4-5-20251001-v1:0` ), others don't ( `global.anthropic.claude-sonnet-4-6` ). Drop or add a piece and you get a model-not-found style error that doesn't explain what's wrong.

## The architecture




**The same agent-with-tools pattern, packed into one function.** Tools are plain Python functions bundled in the package and run in-process. No queues, no orchestration layer, no sidecars: one function, one agent, one execution environment.

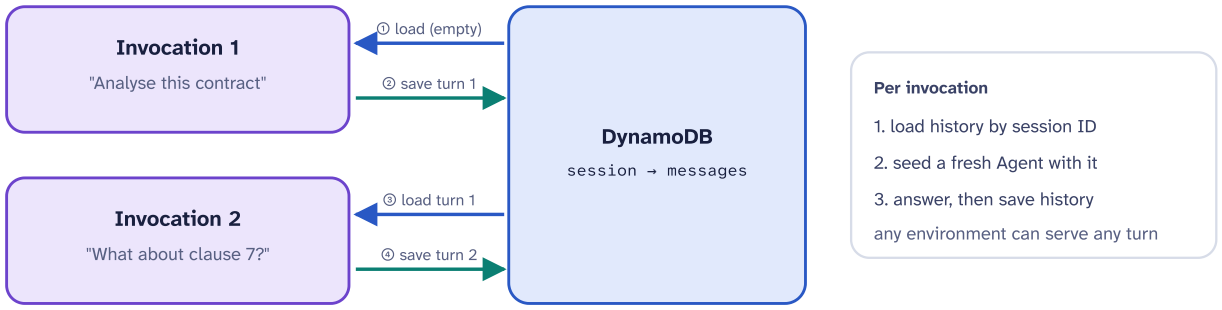
A user sends a query through API Gateway, which invokes the Lambda function. Inside, your agent sits at the centre. It calls out to the LLM for reasoning and runs tools such

as clause extraction or risk scoring. The lab agent has no custom tools at all; the model does the analysis, which keeps the example small.

## Three things that will shape your design

### 1. Statelessness

Each invocation is independent. If the agent asks a clarifying question and the customer replies, that reply is a *new* invocation, possibly on a different environment, and the agent doesn't remember the first one. Unless you persist the conversation somewhere and restore it, your users will complain the agent has amnesia. A common fix is DynamoDB: store the history keyed by a session ID, load it at the start of each invocation, save it at the end.




**The memory lives outside the function.** Lambda environments come and go; the session record in DynamoDB doesn't.

pattern sketch, not part of the lab — illustrative

```
import json
import boto3
from strands import Agent

table = boto3.resource("dynamodb").Table("agent-sessions")
# model and SYSTEM_PROMPT are defined at module level, as in app.py

def lambda_handler(event, context):
    body = json.loads(event["body"])
    session_id = body["session_id"]
```

```
    # 1. Load this session's history (empty on the first turn)
    item = table.get_item(Key={"session_id": session_id}).get("Item",
    history = json.loads(item.get("messages", "[]"))

    # 2. Seed a fresh agent with it, then answer
    agent = Agent(model=model, system_prompt=SYSTEM_PROMPT, messages=h
    result = agent(body["message"])

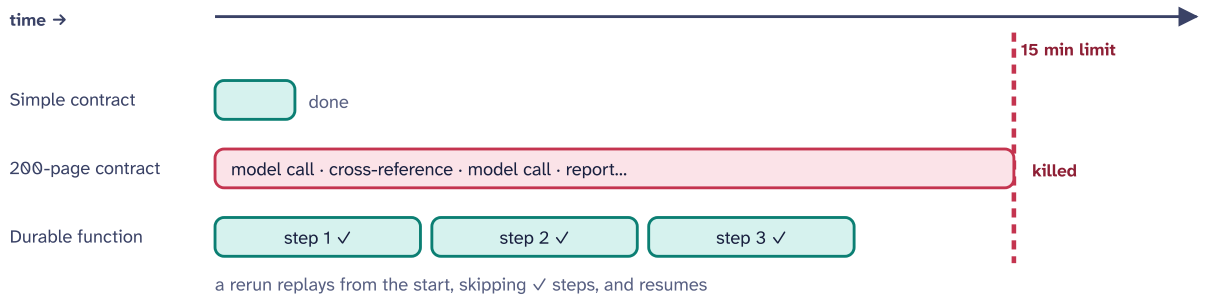
    # 3. Save the updated history for the next invocation
    table.put_item(Item={"session_id": session_id, "messages": json.du
    return {"statusCode": 200, "body": json.dumps({"reply": str(result
```

Keep an eye on size: a single DynamoDB item can hold at most 400 KB, and long conversations about long contracts can exceed it. Strands' built-in session managers (for example one that stores sessions in S3) are an alternative.

### 2. Cold starts

Monday morning, nobody has used the agent since Friday. A customer uploads a contract and waits an extra few seconds (typically 2 to 5, and up to about 15 for a large container image like this lab's) while Lambda starts a fresh environment and loads your dependencies. That's a cold start. It happens on the first invocation and whenever Lambda scales out new environments during a spike. Container images make it slightly slower than zips. Provisioned concurrency keeps a set number of environments pre-warmed, but then you're paying for idle compute, which undercuts one of Lambda's main advantages. Try it in the simulation above.

### 3. The execution ceiling




**Fifteen minutes is plenty, until it isn't.** Most single-agent tasks fit easily; multi-agent workflows with retries and slow calls can hit the wall.

A month in, a customer uploads a 200-page contract. The agent calls the model many times, cross-references clauses and builds a detailed report. Twelve minutes in, it's still going; at 15 minutes Lambda stops the invocation. Lambda durable functions (announced at re:Invent 2025) use checkpoint-and-replay: the function reruns from the start but skips completed checkpoints, resuming where it left off. That works, but it adds complexity to your design.

> ! **In the lab, the real limit is 29 seconds**
>
> The lab puts Lambda behind an API Gateway REST API, which times out after **29 seconds** by default, long before Lambda's 15 minutes. Keep requests short, use a faster model ( `global.anthropic.claude-haiku-4-5-20251001-v1:0` ), or use a Lambda Function URL or AgentCore Runtime for long-running agents.

> → **When Lambda fits**
>
> Lambda works well when your agent responds to an event, does its work and finishes. If it needs to stay running, hold persistent connections, or coordinate a team of agents in a long process, you need something with more staying power. That's next.

## Lab: a contract-analysis agent on Lambda

A serverless contract-analysis agent behind API Gateway: POST a document, get back key terms, flagged risks and a summary. `User → API Gateway → Lambda (container image) → Strands agent → Amazon Bedrock`.

**1** **Check the prerequisites**

`aws sts get-caller-identity` (credentials), `sam --version` (SAM CLI) and `docker info` (Docker running).

**2** **Deploy**

The script checks prerequisites, detects your CPU architecture (arm64 for Apple Silicon or Graviton, x86_64 otherwise), builds the image with `sam build` and deploys with `sam deploy`. It takes 2–3 minutes and prints your endpoint.

**3** **Test with curl**

POST a contract to the endpoint. The first request is slower because of the cold start (5–15 seconds).

**4** **Clean up the same day**

`./cleanup.sh` deletes the whole stack.

lambda/deploy.sh · settings and architecture detection Copy

```
#!/bin/bash
set -e

STACK_NAME="document-analysis-agent"
REGION="${AWS_REGION:-${AWS_DEFAULT_REGION:-us-east-1}}"
MODEL_ID="${BEDROCK_MODEL_ID:-global.anthropic.claude-sonnet-4-6}"

# Auto-detect architecture: arm64 for Apple Silicon / Graviton, x86_64
MACHINE_ARCH=$(uname -m)
if [ "$MACHINE_ARCH" = "arm64" ] || [ "$MACHINE_ARCH" = "aarch64" ]; t
  LAMBDA_ARCH="arm64"
```

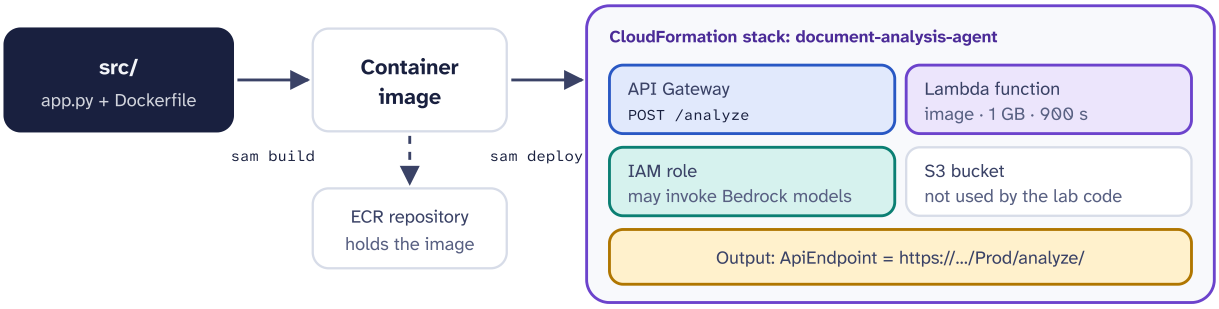
```
else
  LAMBDA_ARCH="x86_64"
fi
```

**lambda/deploy.sh · build and deploy** Copy

```
# --- Step 1: Build ---
echo "Step 1: Building container image (this takes 1-2 minutes the fir
# Clean previous build artifacts to avoid cache issues
rm -rf .aws-sam 2>/dev/null || true
sam build
echo ""

# --- Step 2: Deploy ---
echo "Step 2: Deploying to AWS (this takes 2-3 minutes)..."
sam deploy \
  --stack-name $STACK_NAME \
  --resolve-s3 \
  --resolve-image-repos \
  --capabilities CAPABILITY_IAM \
  --region $REGION \
  --parameter-overrides Architecture=$LAMBDA_ARCH ModelId=$MODEL_ID \
  --no-confirm-changeset \
  --no-fail-on-empty-changeset
```

`--resolve-s3` and `--resolve-image-repos` let SAM create the S3 bucket and container repository it needs; `CAPABILITY_IAM` allows it to create the IAM role; `--parameter-overrides` passes the detected architecture and your model into the template's parameters.




**One stack, one delete.** Because everything belongs to one CloudFormation stack, `sam delete` in `cleanup.sh` removes it all. (Leave the S3 bucket empty: a bucket with files in it can't be deleted.)

terminal Copy

```
cd lambda
chmod +x deploy.sh cleanup.sh
./deploy.sh

curl -X POST <ApiEndpoint printed by deploy.sh> \
  -H "Content-Type: application/json" \
  -d '{"document_text": "This agreement is entered into by Party A and

./cleanup.sh      # when you're done
```

response illustrative; the wording varies

```
{"analysis": "**Parties:** Party A (supplier) and Party B (buyer).
**Key terms:** Party A must deliver 1,000 units by March 2027.
**Risky clause:** \"Late delivery incurs unlimited liability\" exposes
Party A to
uncapped damages. Consider a liability cap or liquidated damages.
**Summary:** A simple supply agreement with one high-risk liability
clause."}
```

> ! **The demo API has no authentication**
>
> Anyone with the URL can call it and spend your Bedrock budget. Don't share the URL, and run `./cleanup.sh` the same day. For real use, add an API key and usage plan, IAM auth, or a Cognito authoriser.

---

## 6.4

# Amazon ECS: always-on and container-driven

A nurse walks up to a scheduling terminal at 2 AM and types: *"Is Dr. Arti available Monday for a knee replacement?"* The agent checks the calendar and offers three open slots. The nurse replies: *"The 9 AM slot works, patient ID is P-1234."* The agent books it, confirms equipment for OR-3 and sends notifications.

Two messages, and the second only makes sense because the agent remembers the first. This isn't a one-off either: staff use the system around the clock, and a conversation might run three, four or five messages before a procedure is booked.

You *could* build this on Lambda: persist the conversation in DynamoDB, reload it on every invocation, rebuild the agent's context each time. It would work, but you'd be fighting the infrastructure instead of building the feature. A Lambda function wants to wake up, do a thing and go away. This agent needs to be awake, warmed up and ready at all times.

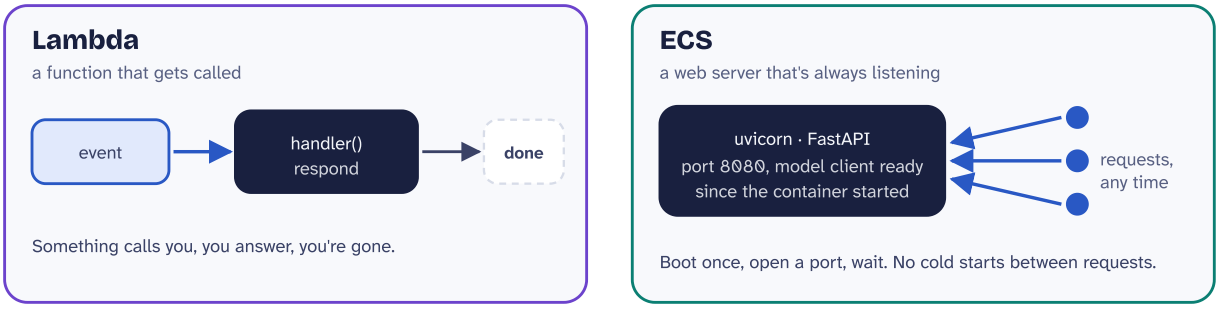
Amazon ECS (Elastic Container Service) gives you that. You put your agent in a Docker container, hand it to ECS and say "keep this running". Run it as an ECS service and it stays up. If the container crashes at 3 AM, the service notices and starts a new one. You don't get paged. You don't write recovery scripts.

**A What's a container?**

A container packs your code together with everything it needs to run (Python, libraries, settings) into one sealed image, like a shipping container that fits on any ship, train or truck. Build it once with `docker build` and it runs the same on your laptop and on AWS.

## From Lambda handler to web server

The mental shift: **on Lambda, your agent is a function that gets called. On ECS, your agent is a web server that's always listening.**




**Called vs listening.** The model client and any expensive setup are created once when the container starts and live for the life of the container.

Here's the course's scheduling agent. It's a FastAPI web app with a Strands agent inside.

`ecs-fargate/app/app.py · the app, model ID and instructions` Copy

```
import os
from pathlib import Path
from fastapi import FastAPI, HTTPException
from fastapi.responses import PlainTextResponse, FileResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
from strands import Agent, tool
from strands.models.bedrock import BedrockModel
```

```
app = FastAPI(title="Hospital Scheduling Agent")

# Serve the web UI
STATIC_DIR = Path(__file__).parent / "static"
app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="sta

MODEL_ID = os.environ.get("MODEL_ID") or os.environ.get("BEDROCK_MODEL

SYSTEM_PROMPT = """You are a hospital surgical scheduling coordinator.
find available time slots for procedures by checking provider availabi
equipment needs, and operating room schedules.

When a scheduling request comes in:
1. Identify the procedure type and required specialists
2. Check provider availability across calendars
3. Verify equipment availability for the requested time
4. Suggest the best available slots
5. Confirm the booking details

Be concise and professional. Present scheduling options clearly with d
times, and any constraints. If conflicts exist, explain them and offer
```

ecs-fargate/app/app.py · a mock tool Copy

```
# --- Mock tools (replace with real integrations in production) ---

@tool
def check_provider_availability(provider_name: str, date: str) -> dict
    """Check a healthcare provider's calendar availability for a given

    Args:
        provider_name: Name of the doctor or specialist
        date: Date to check in YYYY-MM-DD format

    Returns:
        Dictionary with available time slots
    """
    # In production, this would query the EHR/calendar system
    return {
        "provider": provider_name,
        "date": date,
        "available_slots": ["09:00-10:00", "11:00-12:00", "14:00-15:30
```

```
        "notes": "No conflicts found"
    }
```

▶ **See the other two mock tools** ( `check_equipment_availability` , `book_procedure` )

The tools return fixed mock data (three slots, OR-3, booking reference `BK-2026-04821` ). In production each would query the EHR, the calendar system and the booking system. The agent code wouldn't change.

**ecs-fargate/app/app.py · shared model, request shape, fresh agent with history** Copy

```
# Shared model instance (stateless, safe to reuse across requests)
model = BedrockModel(model_id=MODEL_ID)

AGENT_TOOLS = [check_provider_availability, check_equipment_availabili


class HistoryMessage(BaseModel):
    role: str
    content: str


class ScheduleRequest(BaseModel):
    message: str
    history: list[HistoryMessage] = []


def build_agent_with_history(history: list[HistoryMessage]) -> Agent:
    """Create a fresh Agent seeded with prior conversation turns."""
    agent = Agent(
        model=model,
        system_prompt=SYSTEM_PROMPT,
        tools=AGENT_TOOLS,
    )
    # Seed the agent's message buffer with previous turns so it
    # remembers everything the user already said in this session.
    for msg in history:
```

```
            agent.messages.append({"role": msg.role, "content": [{"text":
    return agent
```

ecs-fargate/app/app.py · the endpoints Copy

```
@app.get("/health")
def health_check():
    """Health check endpoint for the ALB."""
    return {"status": "healthy"}


@app.get("/")
def root():
    """Serve the web UI."""
    return FileResponse(str(STATIC_DIR / "index.html"))


# A plain (sync) endpoint: FastAPI runs it in a worker thread, so a sl
# agent call doesn't block other requests.
@app.post("/schedule")
def schedule(request: ScheduleRequest):
    """Send a scheduling request to the hospital agent."""
    if not request.message:
        raise HTTPException(status_code=400, detail="No message provid

    try:
        agent = build_agent_with_history(request.history)
        response = agent(request.message)
        return PlainTextResponse(content=str(response))
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


if __name__ == "__main__":
    import uvicorn
    port = int(os.environ.get("PORT", "8080"))
    uvicorn.run(app, host="0.0.0.0", port=port)
```

1 **`GET /health`**
A tiny endpoint the load balancer and Docker call to check the container is alive. If it stops answering, the container is replaced.

2 **`GET /`**
Serves the chat page ( `static/index.html` ).

3 **`POST /schedule`**
Builds a fresh agent seeded with the conversation so far, asks it the new message and returns plain text. It's a plain (not `async` ) function on purpose: FastAPI runs it in a worker thread, so one slow agent call doesn't block other users.

## Who remembers the conversation?

Each request carries `request.history` : the full conversation so far. The **browser** keeps track of everything the user and agent have said and sends it all with every new message. The server seeds a fresh agent with that history before processing the latest message. That's how, when the nurse says "the 9 AM slot works", the agent still knows it's about Dr. Arti and a knee replacement.

**ecs-fargate/app/static/index.html · the browser keeps the history** Copy

```
// Conversation history sent with every request so the agent
// remembers what the user said in previous turns.
const history = [];
…
  try {
    const res = await fetch('/schedule', {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({ message: msg, history: history })
    });
    thinking.remove();
    if (res.ok) {
```

```
const text = await res.text();
addMsg(text, 'agent');
// Append both turns so the next request carries full context
history.push({ role: 'user', content: msg });
history.push({ role: 'assistant', content: text });
…
```

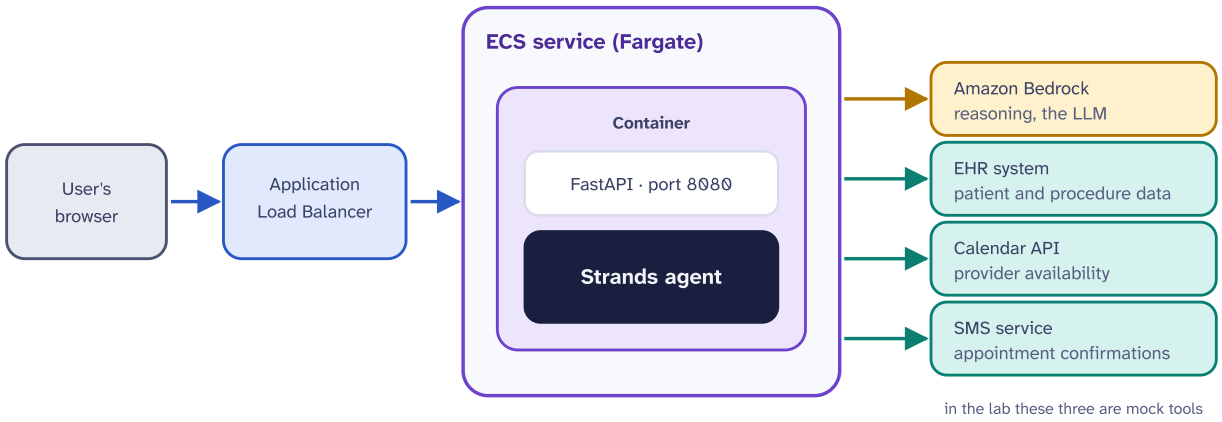
This is the simplest approach, not the only one. Strands has built-in session management that can persist conversations to the local filesystem or to Amazon S3, and you can write a custom session repository backed by DynamoDB or any database. Those are better for production because the *server* owns the state, so it survives page refreshes and can be audited. The browser-carries-history pattern suits demos and light deployments where you don't want a persistence layer.

**Try it** **The 2 AM booking, three ways**

Send message 1, then message 2. Then try refreshing the browser between them.

## What's actually happening on AWS

You don't need to be a networking expert to understand the ECS deployment model. Here's what it looks like:



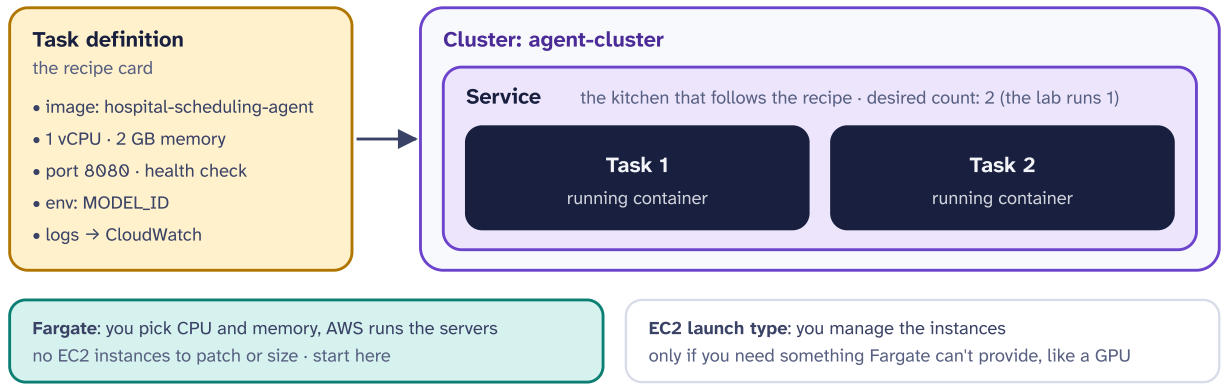

**Browser → load balancer → container → model and tools.** The load balancer is the front door; the ECS service keeps the containers behind it alive.

A user opens a URL. That request hits an Application Load Balancer, which routes traffic only to healthy containers. Run two containers and one dies: the load balancer sends everything to the survivor while ECS starts a replacement. Users don't notice.

Behind it, the **ECS service** keeps your containers running. You tell it "I want two copies of my agent running at all times", and it makes that happen. Crash? Replaced. New version? The service rolls it out gradually, keeping the old version running until the new one passes its health checks.

The lab runs a single task ( `DESIRED_COUNT=1` ) to keep costs down, so a crash there means a short outage until ECS replaces it; raise the count to 2 or more when you need failover.

Each container runs from a task definition: a config file that says "use this Docker image, give it this much CPU and memory, send its logs to CloudWatch".




**Recipe, kitchen, dishes.** The task definition describes one container; the service keeps the right number of them running inside a cluster.

For the compute you have two options. With AWS Fargate you specify CPU and memory and AWS handles the servers: you never see an EC2 instance, never patch an operating system, never think about capacity. With the **EC2 launch type** you manage the underlying instances yourself, which you'd only do for something Fargate can't provide, like a GPU. Start with Fargate; switch later if you need to.

## Lab: a hospital scheduling agent on ECS Fargate

A Strands agent served by FastAPI on ECS Fargate behind an Application Load Balancer, with a small chat UI. It isn't production-hardened (mock tools, no authentication, minimal error handling), but it walks through every step from Python code on your laptop to a running agent behind a public URL, and you can read the deploy script to understand each step.

```
architecture (from the lab README)

Browser (chat UI) → ALB (port 80) → ECS Fargate task → Amazon Bedrock
                                          |
                                   FastAPI + Strands agent
                                   (tools: provider calendar,
                                    equipment check, booking)
```

### Prerequisites

AWS credentials ( `aws sts get-caller-identity` ), Docker running ( `docker info` ), AWS CLI v2, and a default VPC with at least two public subnets (new accounts have one).

## The container image

ecs-fargate/Dockerfile Copy

```
FROM public.ecr.aws/docker/library/python:3.12-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    curl \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ .

RUN useradd -m appuser
USER appuser

EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080", "--w
```

| Line | Why |
| --- | --- |
| `FROM … python:3.12-slim` | A small official Python base image (pulled from AWS's public registry to avoid Docker Hub rate limits) |
| `apt-get install curl` | Needed by the health check command |

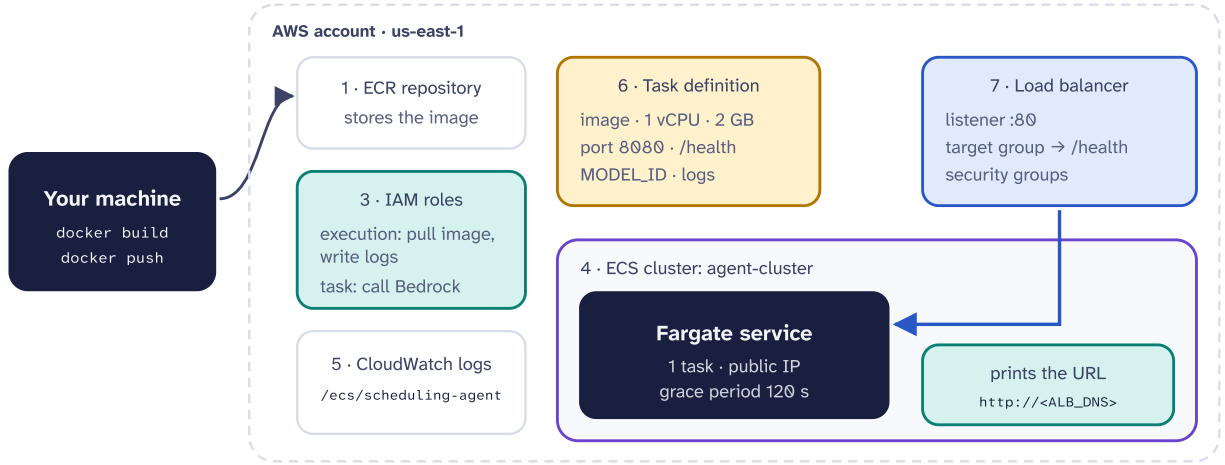
| Line | Why |
| --- | --- |
| `pip install … then COPY app/` | Dependencies first, code second, so code changes don't reinstall every package |
| `useradd … USER appuser` | Don't run as root: if the app is ever compromised, the attacker isn't root in the container |
| `HEALTHCHECK … /health` | Docker checks the app every 30 seconds when you `docker run` it locally. ECS ignores this line and runs the identical check from the task definition (step 6), alongside the load balancer's own check. |
| `uvicorn … --workers 2` | uvicorn, the web server, runs two worker processes so requests are served in parallel |

## Run it locally first (optional)

terminal Copy

```
cd ecs-fargate
pip install -r requirements.txt
cd app && uvicorn app:app --port 8080
# open http://localhost:8080
```

## What `deploy.sh` does, step by step




**Seven steps, all visible in one script.** Every `aws` command in `deploy.sh` is something a platform team would otherwise do in the console.

ecs-fargate/deploy.sh · configuration Copy

```
# ── Configuration ──────────────────────────────────────────────
APP_NAME="hospital-scheduling-agent"
CLUSTER_NAME="agent-cluster"
SERVICE_NAME="scheduling-agent-service"
TASK_FAMILY="scheduling-agent"
CONTAINER_PORT=8080
CPU=1024          # 1 vCPU
MEMORY=2048       # 2 GB
DESIRED_COUNT=1
MODEL_ID="${BEDROCK_MODEL_ID:-global.anthropic.claude-sonnet-4-6}"

REGION="${AWS_REGION:-${AWS_DEFAULT_REGION:-$(aws configure get region
REGION="${REGION:-us-east-1}"
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text
ECR_REPO="${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/${APP_NAME}"
IMAGE_TAG="latest"

# Who may reach the load balancer. Default: only your current public I
```

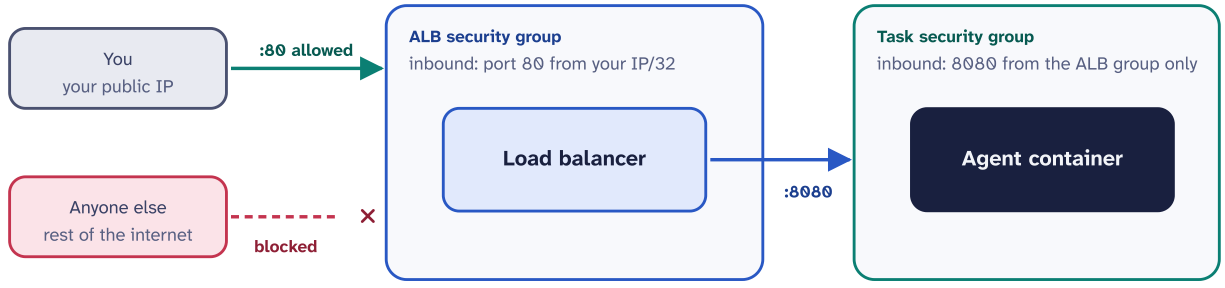
```
# For a live class demo you can open it to everyone: ALB_INGRESS_CIDR=
if [ -z "${ALB_INGRESS_CIDR:-}" ]; then
  MY_IP=$(curl -s --max-time 5 https://checkip.amazonaws.com | tr -d '
  if [ -n "${MY_IP}" ]; then ALB_INGRESS_CIDR="${MY_IP}/32"; else ALB_
fi
```

**ecs-fargate/deploy.sh · step 6, the task definition** Copy

```
# ── Step 6: Register task definition ──────────────────────────
echo "Step 6/7: Registering task definition..."
TASK_DEF=$(cat <<EOF
{
  "family": "${TASK_FAMILY}",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "${CPU}",
  "memory": "${MEMORY}",
  "executionRoleArn": "${EXEC_ROLE_ARN}",
  "taskRoleArn": "${TASK_ROLE_ARN}",
  "containerDefinitions": [
    {
      "name": "${APP_NAME}",
      "image": "${ECR_REPO}:${IMAGE_TAG}",
      "essential": true,
      "portMappings": [{"containerPort": ${CONTAINER_PORT}, "protocol"
      "environment": [
        {"name": "MODEL_ID", "value": "${MODEL_ID}"}
      ],
      "healthCheck": {
        "command": ["CMD-SHELL", "curl -f http://localhost:${CONTAINER
        "interval": 30,
        "timeout": 5,
        "retries": 3,
        "startPeriod": 60
      },
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "${LOG_GROUP}",
          "awslogs-region": "${REGION}",
          "awslogs-stream-prefix": "agent"
        }
      }
```

```
      }
    ]
}
EOF
)
echo "${TASK_DEF}" > /tmp/task-def.json
aws ecs register-task-definition --cli-input-json file:///tmp/task-def
  --region "${REGION}" --output text --query 'taskDefinition.taskDefin
rm /tmp/task-def.json
echo "  Task definition: ${TASK_FAMILY}"
```

`networkMode: awsvpc` gives each task its own network address (required for Fargate). There are two roles: the **execution role** is for ECS itself (pull the image from ECR, write logs), the **task role** is for your code (call Bedrock). Keeping them separate is least privilege in practice. `startPeriod: 60` gives the app a minute to boot before failed health checks count.




**Two locked doors.** Only you can reach the load balancer, and only the load balancer can reach the container.

The addresses use CIDR notation: `203.0.113.7/32` means exactly that one address, `0.0.0.0/0` means every address. The script writes the load balancer's rule only when it first *creates* the security group, so `ALB_INGRESS_CIDR` only takes effect on a first deploy. To change it later (or if your own IP changes), run `./cleanup.sh` and deploy again, or add a rule for your new IP to the load balancer's security group. Inside the load balancer, a listener on port 80 forwards requests to a target group: the set of containers, checked through `/health`.

terminal · deploy, test, watch logs, clean up Copy

```
cd ecs-fargate
chmod +x deploy.sh cleanup.sh
./deploy.sh                                          # 3-4 min, then 2-3 min

# OR, on your first deploy, open the load balancer to everyone for a l
ALB_INGRESS_CIDR=0.0.0.0/0 ./deploy.sh

curl http://<ALB_DNS>/health
curl -X POST http://<ALB_DNS>/schedule -H 'Content-Type: application/j
  -d '{"message": "Schedule a knee arthroscopy for patient P-1234 with

aws logs tail /ecs/scheduling-agent --follow --region us-east-1

./cleanup.sh
```

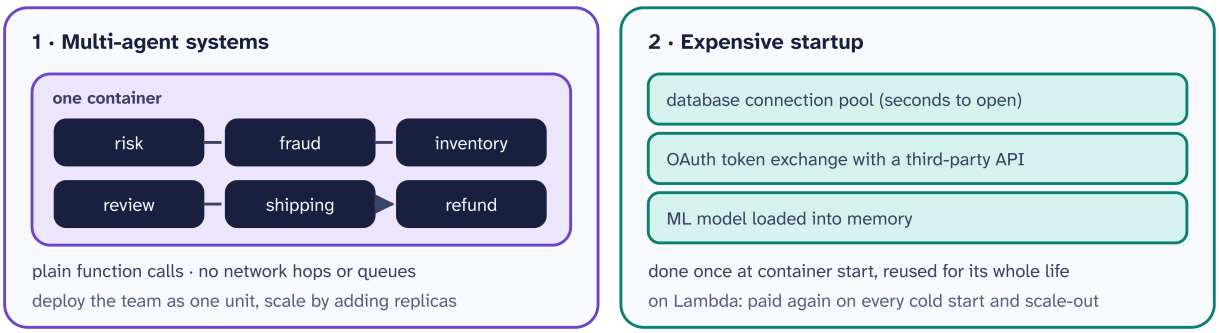
Open the URL printed at the end and you'll see the chat UI. Type *"Is Dr. Smith available next Tuesday for a knee replacement?"*, then follow up with just a time slot and a patient ID. The agent connects the dots across messages. The point isn't the scheduling app; it's seeing the whole journey from Python on your laptop to an agent behind a public URL on AWS.

```
POST /schedule                                   illustrative; the wording varies

I've checked Dr. Smith's calendar for 2026-11-18. Available slots:
  • 09:00–10:00   • 11:00–12:00   • 14:00–15:30
The arthroscopy tower is available in OR-3 for the 09:00 slot.
Shall I book patient P-1234 for 09:00–10:00 in OR-3?
```

> ! **Cost: clean up the same day**
>
> About $0.05 per hour for one 1 vCPU / 2 GB task, plus the load balancer's hourly charge, whether or not anyone uses it. `./cleanup.sh` deletes the service, load balancer, target group, task definitions, cluster, ECR repository, log group, security groups and IAM roles. The README covers networking details, IAM and troubleshooting if you want to dig deeper.

## When to reach for Amazon ECS




**Teams in one box, and setup paid once.** These are the two situations where an always-on container clearly beats a function.

**Multi-agent systems.** The e-commerce order-processing graph from Section 4 had specialised agents for risk, fraud, inventory, shipping and more. On ECS, all of them live in the same container and talk through plain function calls: no network hops, no serialisation overhead, no message queues between them. You deploy the whole team as one unit and scale by adding replicas.

**Expensive startup.** Database connection pools that take seconds to open, authenticated sessions with third-party APIs that involve OAuth token exchanges, ML models loaded into memory: on ECS that happens once and persists for the life of the container. On Lambda you'd pay some of it on every cold start, and all of it whenever Lambda scales up new environments during a spike.

---

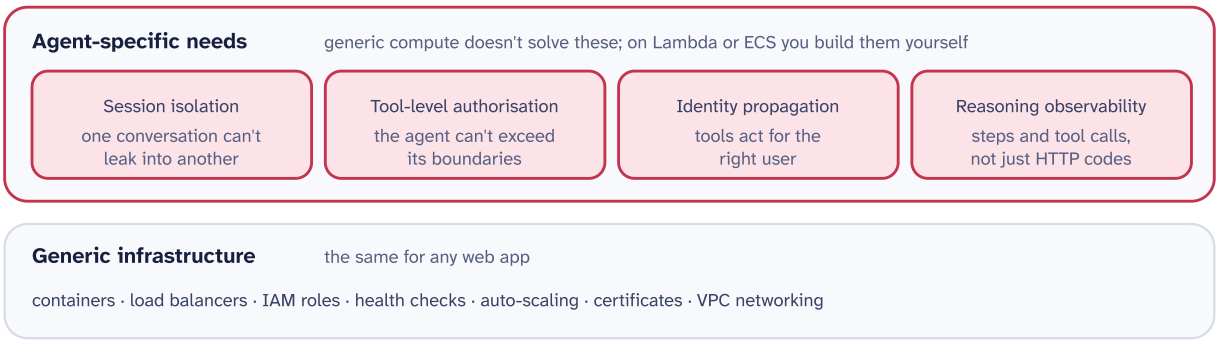
## 6.5 Amazon Bedrock AgentCore

Deploying on Lambda and ECS meant writing Dockerfiles, configuring load balancers, creating IAM roles, adding health checks and managing task definitions. And those were the *simple* versions.

Productionising adds much more: monitoring, auto-scaling policies, certificates, VPC configuration and a dozen other things that have nothing to do with your agent's actual job. None of it is agent-specific: you'd do exactly the same for a REST API, a batch

processor or a website. The infrastructure doesn't know or care that it's running an AI agent.

But agents have problems that generic compute doesn't solve for you:

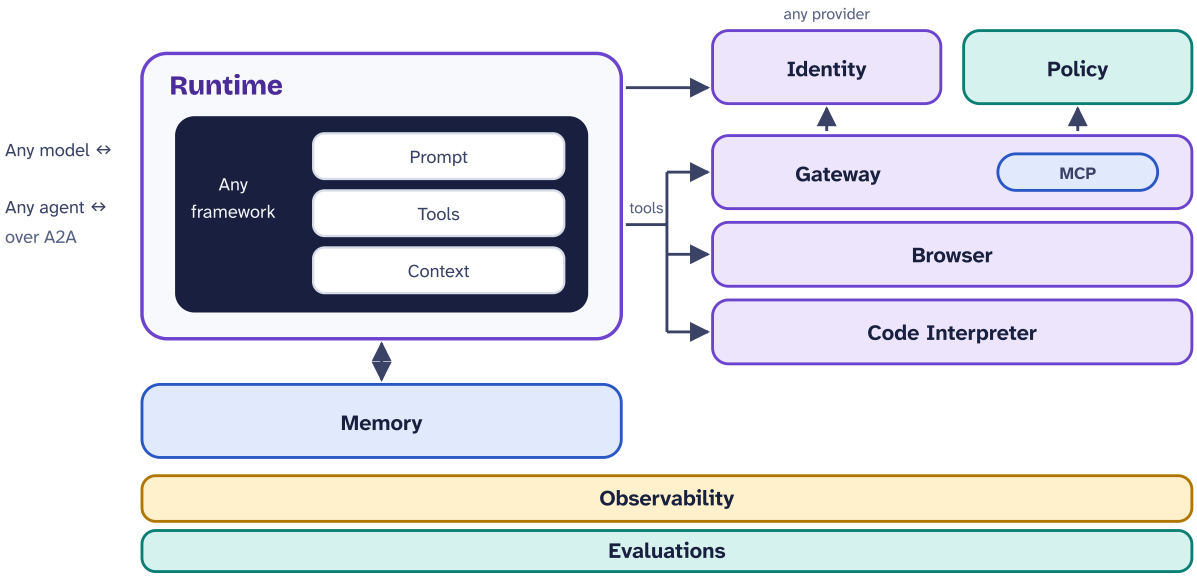



**The part nobody warns you about.** Generic infrastructure is well understood; the agent-specific layer on top is where teams lose weeks.

This is where Amazon Bedrock AgentCore comes in. In Section 1 you met it in the middle layer of the AWS AI stack, the managed solutions alongside Amazon Bedrock, between ready-made applications above and do-it-yourself infrastructure below. In Section 3 you used AgentCore Memory for short- and long-term memory. Those were individual services solving individual problems. Now you'll see how the full set fits together.

## The AgentCore architecture

AgentCore is a set of **composable services**, each solving a specific challenge every team hits when moving agents from prototype to production. Use them all together, or pick the ones you need.




**Every box solves a problem you'd otherwise build yourself.** Understanding the architecture is what turns AgentCore from "another managed service" into a genuine shortcut to production.

| Service | What it does | Covered |
|---|---|---|
| **Runtime** | Where the agent actually runs. Framework-agnostic (Strands, LangGraph, CrewAI, LlamaIndex, OpenAI Agents SDK, Google ADK, Claude Agent SDK or your own) and model-agnostic (Bedrock, OpenAI, Anthropic, Google...). Can expose the agent over A2A so agents on other platforms can find and talk to it. | 6.6 |
| **Gateway** | Turns enterprise APIs (OpenAPI specs, Smithy models, Lambda functions, remote MCP servers) into MCP tools the agent can discover and call. No integration code per API. | 6.8 |
| **Browser** | A managed, sandboxed web browser the agent can use to interact with web pages | – |
| **Code Interpreter** | An isolated sandbox for running Python, JavaScript or TypeScript the agent writes. Browser and Code Interpreter show up as ordinary tools. | – |

| Service | What it does | Covered |
|---|---|---|
| **Identity** | Authentication in both directions: users to your agent (Cognito, Okta, Entra ID, any OAuth provider), and your agent to external services on behalf of those users | 6.9 |
| **Policy** | Deterministic rules written in Cedar that intercept every tool call before it runs. "No refunds over $1,000" is enforced by the infrastructure, not the agent's reasoning, so no prompt can talk its way around it. | Section 7 |
| **Memory** | The short- and long-term memory service from Section 3, wired directly into Runtime | Section 3 |
| **Observability** | Captures reasoning steps, tool calls, model calls, policy decisions, latency and errors across every service. Built on OpenTelemetry, so it plugs into CloudWatch, Datadog, Grafana, Splunk and others. | Section 7 |
| **Evaluations** | LLM-as-judge scoring, built-in evaluators (faithfulness, relevance) and continuous online evaluation in production | Section 7 |

**Think of it like this**

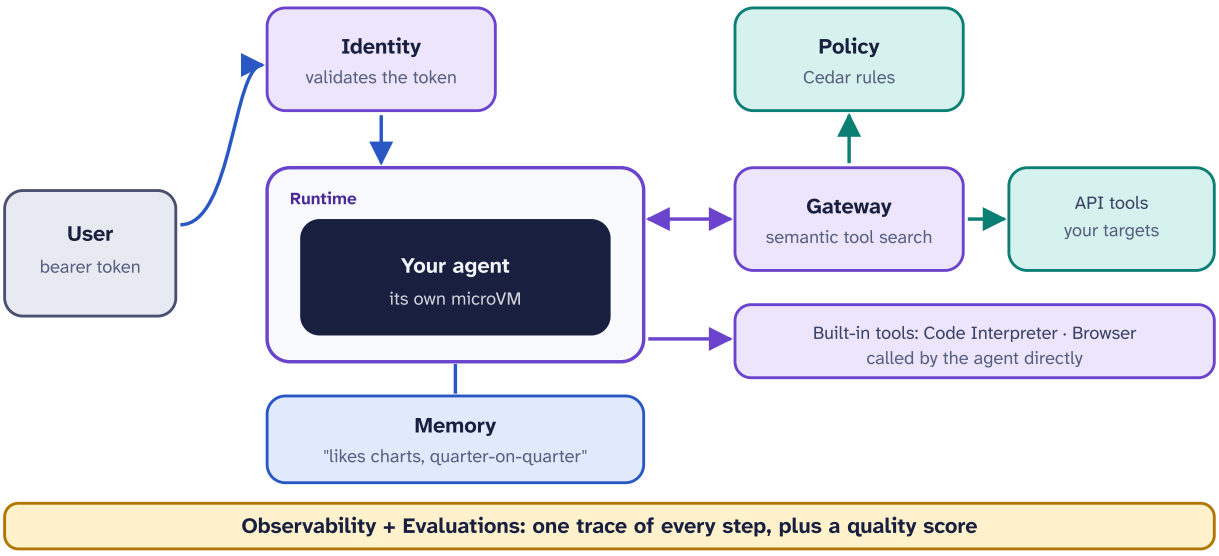
AgentCore is a serviced office building for agents. Runtime is the private office (one per visitor, cleaned after each meeting). Identity is the front desk checking badges. Policy is the building's rules that security enforces no matter what anyone says. Gateway is the switchboard that connects you to every department. Memory is the filing cabinet, Observability is the CCTV, and Evaluations is the performance review. Rent one office, or take the whole floor.

**The services are composable.** Run your agent on ECS but use Gateway for tools. Use Runtime for hosting but manage your own authentication. Use just Memory if that's all you need. Each service has its own API, so you can adopt them independently. Used together they're integrated: Identity flows through to Gateway, Runtime connects to

Memory, and Observability captures the full picture without you instrumenting anything.

## One request, end to end

Here's how the pieces work together. A user asks your agent: *"Analyse this sales data and create a summary report."* Step through it:




1 Authenticate | 2 Agent + memory | 3 Find tools | 4 Policy check

5 Run tools | 6 Update memory | 7 Trace + score

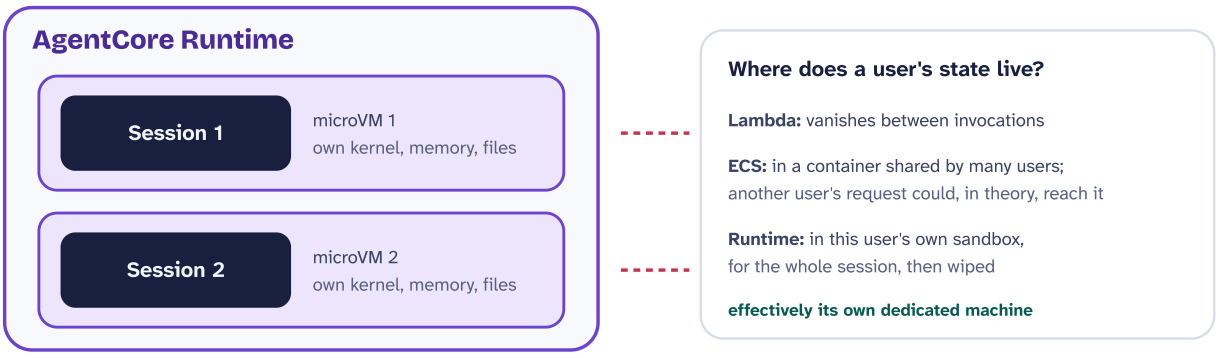
Click step 1 to begin.

**The user sees a report with charts.** Behind it, the AgentCore stack coordinates securely, with governance and full traceability. On Lambda or ECS you'd build most of this coordination yourself.

# 6.6 AgentCore Runtime

Runtime is where the agent lives and executes. Its defining feature: each user session gets its own microVM.

Inside that dedicated environment the agent can keep state, hold database connections and build up context across many turns. When the session ends, the whole microVM is terminated and its memory sanitised. Nothing is shared between sessions, so nothing can leak between them. In financial services, for example, an agent processing one client's portfolio has zero chance of exposing it to another client's session.




**One microVM per session.** A microVM is a tiny virtual machine that starts in a fraction of a second but has the hard isolation of a full VM.

Consider a multi-turn conversation where a user asks the agent to analyse a financial document. The agent extracts data, stores intermediate results and writes temporary files. On Lambda all of that vanishes between invocations. On ECS it lives in a shared container. On AgentCore each user gets their own sandbox, and the agent can behave as if it's running on a dedicated machine.

## Session states, and what you pay for

When a user sends a request, AgentCore creates a session and starts a microVM. The session moves through three states: **Active** while the agent is processing, **Idle** while it waits for the next user message, and **Terminated** when it ends. The default idle timeout is **15 minutes**: if the user goes quiet longer than that, the session ends and

the microVM is cleaned up. A session can live for up to **8 hours** in total; both limits are configurable.

The cost model is what makes this attractive. Runtime bills per second for the CPU and memory your session actually consumes. While the agent is *waiting* (for the user, for the model's reply, for a tool), CPU usage drops to zero, so in a typical support conversation where the user takes a minute between messages you pay CPU for the 10–20 seconds of real processing per turn, not for the silence. Memory is billed for as long as the session stays open, idle time included, which is why you stop sessions you've finished with (or let the idle timeout do it). Your session state is kept throughout.

Try it **A session's life, and its compute bill**

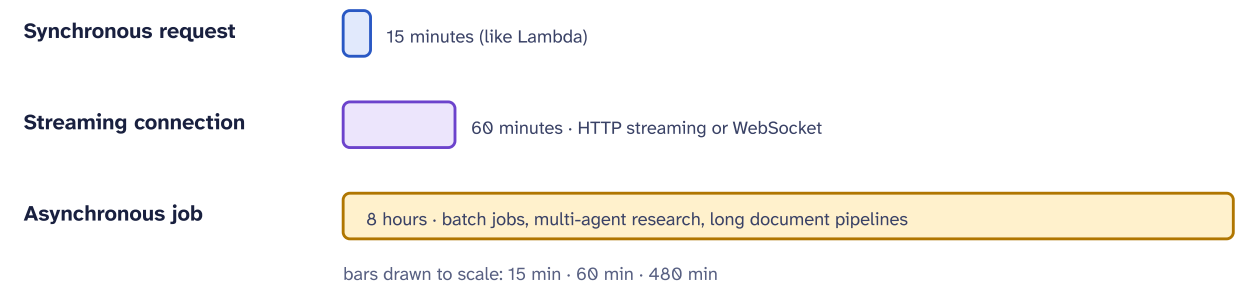
No session | Active | Idle | Terminated

Active (agent working) | Idle (waiting, state kept) | Terminated

| SESSION CLOCK | CPU-BUSY TIME | SHARE OF TIME WITH CPU BILLED | AN ALWAYS-ON CONTAINER BILLS CPU FOR |
|---|---|---|---|
| 0 s | 0 s | – | 0 s |

Start by sending a message. Timings are illustrative.

## How long can a session run?




**The pattern sets the ceiling.** For long jobs the agent tells AgentCore it's busy, not idle, so the session isn't cleaned up mid-task.

If your agent kicks off a long operation, it reports `HealthyBusy` on its health-check endpoint so AgentCore knows the session is working, not idle. The SDK's `app.add_async_task(...)` and `app.complete_async_task(...)` helpers manage that for you.

## Making an agent Runtime-ready: four additions

agentcore-runtime/agent.py Copy

```python
import os

from bedrock_agentcore import BedrockAgentCoreApp
from strands import Agent, tool
from strands.models.bedrock import BedrockModel

app = BedrockAgentCoreApp()

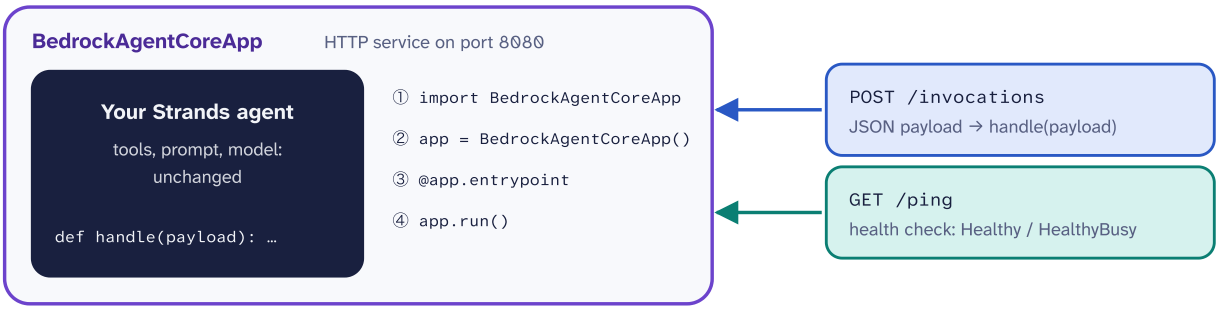
MODEL_ID = os.getenv("BEDROCK_MODEL_ID", "global.anthropic.claude-sonn


@tool
def extract_clauses(text: str) -> dict:
    """Extract key clauses from contract text."""
    # In production this would use NLP or a specialised model.
    # For this example we return a structured placeholder.
    return {"clauses": ["termination", "liability", "indemnification"]
```

```
model = BedrockModel(model_id=MODEL_ID)
agent = Agent(
    model=model,
    tools=[extract_clauses],
    system_prompt="You are a document analysis assistant. Extract and
)


@app.entrypoint
def handle(payload):
    response = agent(payload.get("prompt"))
    return response.message["content"][0]["text"]


if __name__ == "__main__":
    app.run()
```




**No server code to write.** The SDK provides the web server, the routes and the health check.

Only four things change from local development code: import `BedrockAgentCoreApp`, create the `app`, decorate your invocation function with `@app.entrypoint`, and call `app.run()`. Behind the scenes the app becomes an HTTP service with `/invocations` for requests and `/ping` for health checks. The agent logic (tools, system prompt, model) is the same code you've been writing since Section 2.

One subtle point: here the `agent` is created *once, at module level*, which on Lambda would mix up different callers' conversations. On Runtime it's exactly right, because

every session gets its own microVM and therefore its own agent object. Within a session, follow-up messages reach the same agent, so it remembers the conversation.

terminal · test locally Copy

```
pip install bedrock-agentcore strands-agents
python agent.py
# in another terminal
curl -X POST http://localhost:8080/invocations -H "Content-Type: appli
  -d '{"prompt": "What are the key clauses in a standard NDA?"}'
```

response illustrative; the wording varies

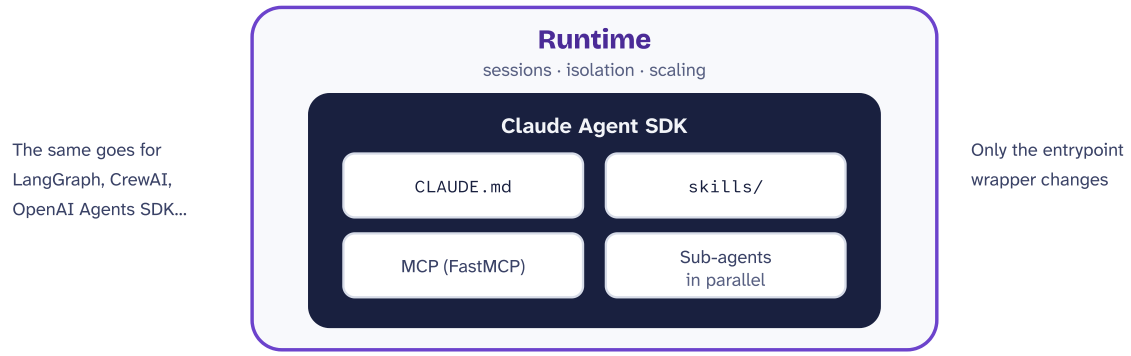
```
"I'd be happy to help, but I'll need the actual contract text to
extract its clauses.
If you're asking generally, the most common NDA clauses are:\n\n1.
**Definition of
Confidential Information**\n2. **Obligations of the Receiving
Party**\n3. **Permitted
Uses**\n4. **Exceptions** (public or independently developed
information)\n5. **Term**
\n6. **Return or Destruction**\n7. **Remedies**\n8. **Governing
Law**\n\nPaste an NDA
and I'll extract its actual key clauses."
```

## Any framework, deployed the same way

Runtime is framework-agnostic. Strands, LangGraph, CrewAI, OpenAI Agents SDK, Google's Agent Development Kit (ADK), the Claude Agent SDK, or something custom: they all deploy the same way. That matters because most organisations end up with agents in several frameworks, through acquisitions, team preferences, or because different frameworks suit different problems.

Say your team builds agents with the Claude Agent SDK and already has a whole programming model: a `CLAUDE.md` with project context, a `skills/` folder of reusable capabilities, MCP servers wired to your tools, maybe sub-agents running jobs in parallel. You can deploy all of that as-is: wrap the entry point with `@app.entrypoint` and add `app.run()`. The SDK handles its own tool discovery,

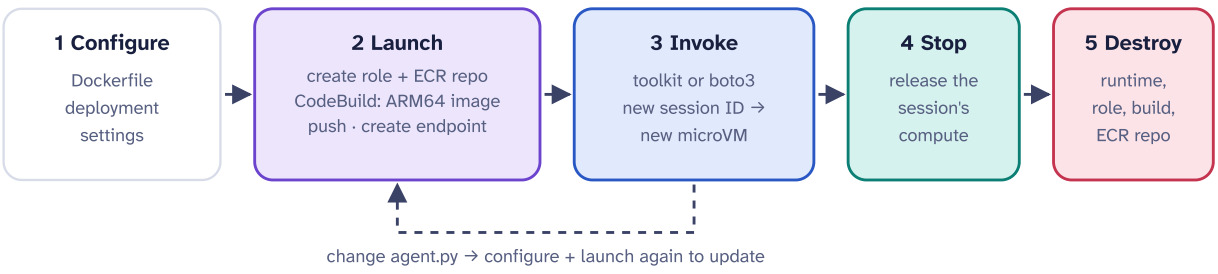
skill loading and sub-agent orchestration inside the microVM. Runtime doesn't know or care what's inside; it manages the session, the isolation and the scaling.




**Bring your whole programming model.** Everything runs inside the microVM; Runtime manages the session around it.

## Lab: deploy the contract agent to AgentCore Runtime

The notebook `agentcore-runtime/deploy_to_agentcore_runtime.ipynb` walks the full lifecycle using the AgentCore starter toolkit's `Runtime` class: configure, launch, invoke, manage sessions, clean up. The build runs in AWS CodeBuild, so you don't need Docker. You need IAM permissions to create IAM roles, ECR repositories, CodeBuild projects, S3 buckets and AgentCore runtimes.




**Five calls from code to cloud and back.** The first launch takes a few minutes; later updates are faster because the toolkit reuses cached dependencies.

### Step 0 and 1: tooling and the agent

notebook · install the deployment tooling on your machine Copy

```
%pip install -q bedrock-agentcore-starter-toolkit 'bedrock-agentcore>=
```

The notebook then writes the same `agent.py` you saw above, and a `requirements.txt` listing what gets installed *inside* the deployed container:

agentcore-runtime/requirements.txt Copy

```
# Packages installed INSIDE the deployed AgentCore container
bedrock-agentcore>=1.24.0,<2.0.0
strands-agents>=1.60.0,<2.0.0
```

Step 2 is the local test you just saw: `python agent.py` in a separate terminal, then `curl` to `/invocations`. Stop it with Ctrl+C when you're happy.

### Step 3: configure

notebook · configure Copy

```
from boto3.session import Session
from bedrock_agentcore_starter_toolkit import Runtime

region = Session().region_name or "us-east-1"
print(f"Deploying to region: {region}")

runtime = Runtime()
runtime.configure(
    entrypoint="agent.py",
    requirements_file="requirements.txt",
    auto_create_execution_role=True,
    auto_create_ecr=True,
    region=region,
    agent_name="doc_analysis_agent",
)
```

`configure` generates a Dockerfile from your code and saves the deployment settings. The two `auto_create_…` flags ask the toolkit to create an IAM execution

role with the permissions your agent needs and an Amazon ECR repository; it does that when you launch.

### Step 4 and 5: launch and wait

notebook · launch Copy

```
import os

# The deployed agent reads BEDROCK_MODEL_ID (see agent.py); pass your
MODEL_ID = os.getenv("BEDROCK_MODEL_ID", "global.anthropic.claude-sonn

launch_result = runtime.launch(auto_update_on_conflict=True, env_vars=

print(f"Agent ID:  {launch_result.agent_id}")
print(f"Agent ARN: {launch_result.agent_arn}")
print(f"ECR URI:   {launch_result.ecr_uri}")
```

`launch` builds the container image in CodeBuild (ARM64), pushes it to ECR and creates the Runtime endpoint. `env_vars` passes your model choice into the deployed container, and `auto_update_on_conflict=True` lets you rerun it to update an existing agent.

notebook · wait until READY Copy

```
import time

status = runtime.status().endpoint["status"]
print(f"Status: {status}")
while status not in ["READY", "CREATE_FAILED", "UPDATE_FAILED"]:
    time.sleep(10)
    status = runtime.status().endpoint["status"]
    print(f"Status: {status}")

print("\nEndpoint is ready." if status == "READY" else "\nDeployment f
```

### Step 6 and 7: invoke it

notebook · invoke with the starter toolkit Copy

```
response = runtime.invoke(
    {"prompt": "Analyze this contract clause: The vendor shall be liab
)
print(response["response"][0] if response.get("response") else respons
```

notebook · invoke with boto3, as another service would Copy

```
import json
import uuid

import boto3

client = boto3.client("bedrock-agentcore", region_name=region)

response = client.invoke_agent_runtime(
    agentRuntimeArn=launch_result.agent_arn,
    runtimeSessionId=str(uuid.uuid4()),
    qualifier="DEFAULT",
    payload=json.dumps({"prompt": "Summarize the liability and indemni
)

session_id = response.get("runtimeSessionId")
print(f"Session ID: {session_id}")

# The body is streamed in chunks
content = [chunk.decode("utf-8") for chunk in response.get("response",
print(f"\nAgent response:\n{json.loads(''.join(content))}")
```

1 **`invoke_agent_runtime`**

How a Lambda function, an ECS service or Step Functions would call your agent in production, the same pattern as calling any other AWS service.

2 **`runtimeSessionId`**

Which session (and so which microVM) the request belongs to. Reuse the same ID for the next message to continue the conversation; a new ID starts a new isolated session. IDs must be at least 33 characters, which a UUID (36) satisfies.

**3** **A streamed body**

The response arrives in chunks; the code joins and decodes them.

### Step 8 and 9: stop the session, then clean up

notebook · stop the session Copy

```
if session_id:
    client.stop_runtime_session(
        agentRuntimeArn=launch_result.agent_arn,
        runtimeSessionId=session_id,
        qualifier="DEFAULT",
    )
    print(f"Session {session_id} stopped.")
```

Each session costs money (vCPU and memory) while it's open. When a conversation is done, stop the session explicitly, or rely on the idle timeout (15 minutes by default) to clean it up.

notebook · destroy everything Copy

```
result = runtime.destroy(delete_ecr_repo=True)
print("Removed:", *result.resources_removed, sep="\n  - ")
if result.errors:
    print("Errors:", result.errors)
```

**>_ Prefer the command line?**

The starter toolkit also installs an `agentcore` CLI. Add `--deployment-type container` to get the same container build as the notebook; without it, the CLI may choose direct code deployment instead.

terminal Copy

```
pip install bedrock-agentcore-starter-toolkit
agentcore configure -e agent.py --deployment-type container --non-inte
```

```
agentcore deploy
agentcore invoke '{"prompt": "Summarize the termination clause risks"}
agentcore destroy
```

> **A notice you may see**
>
> Recent versions of the starter toolkit's CLI print a yellow notice recommending AWS's newer AgentCore CLI (the npm package `@aws/agentcore`, which is also called `agentcore` but has different commands). The course code uses the Python toolkit, which still works; set `AGENTCORE_SUPPRESS_RECOMMENDATION=1` to hide the notice.

## Three ways to get code into Runtime

| Option | How | When |
|---|---|---|
| **Container built for you** | Hand over your source; the toolkit generates a Dockerfile and builds it in the cloud (this lab) | The easy default |
| **Bring your own container** | Push an image your CI/CD pipeline already builds. It must be ARM64 (Graviton), so plan your builds for that. | You already have container pipelines |
| **Direct code deployment** | Upload your Python code as a package, with no container at all | Simple agents, fastest iteration |

▶ **See the Dockerfile the toolkit generates** (kept in the folder for reference)

> **Try next**
>
> Add a tool to `agent.py` and run the configure and launch steps again to update the deployment.

## 6.7

# How Runtime compares to Lambda and ECS

You've now seen all three deployment paths. Here's how they stack up.

| | Lambda | ECS (Fargate) | AgentCore Runtime |
|---|---|---|---|
| **Session model** | Stateless: each invocation is independent | Shared container across requests | Dedicated microVM per user session |
| **State across turns** | Persist externally (DynamoDB), reload every call | In memory within the container | Kept automatically in the microVM |
| **Max execution** | 15 minutes per invocation | No limit (the container stays running) | 15 min sync, 60 min streaming, 8 hours async |
| **Scaling unit** | Per request | Per container (task count) | Per session |
| **Pricing model** | Per invocation + duration | Per allocated vCPU/memory (always on) | Resources actually consumed; CPU stops billing while waiting |
| **Auth, observability, policy** | You build it | You build it | Managed services you configure |
| **Infrastructure control** | Moderate (VPC, layers, concurrency) | Full (networking, security groups, instance types) | Abstracted: less control, less to manage |

The last two rows are usually where the decision is made. Lambda and ECS are general-purpose compute: they'll run your agent, but auth, observability, tool integration and policy enforcement are your responsibility. AgentCore handles those as managed services, which saves weeks of engineering, but you give up fine control over things like network placement, security groups and GPU access. **ECS is the escape hatch** when you need that control.

Try it **Pick the home for each agent**

**1. Summarise every invoice PDF uploaded to S3, about 20 seconds each, with traffic that comes and goes.**

Lambda | ECS | AgentCore Runtime

**2. A support agent where each customer chats over several turns and must never see another customer's data.**

Lambda | ECS | AgentCore Runtime

**3. A team of six agents that call each other with plain function calls, plus a GPU for a local model.**

Lambda | ECS | AgentCore Runtime

**4. Continuous monitoring of factory sensors that never stops and has no user sessions.**

Lambda | ECS | AgentCore Runtime

**5. Score each card transaction as it arrives: one independent decision, scaling to zero at night.**

Lambda | ECS | AgentCore Runtime

**6. A research agent that sometimes needs three hours for a report, and you'd rather not build auth and tracing yourself.**

Lambda | ECS | AgentCore Runtime

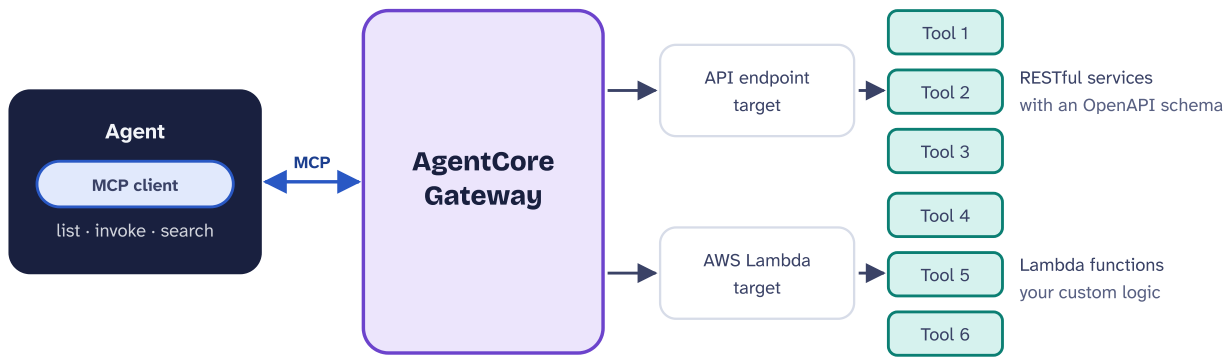
0 of 6 answered

## 6.8 AgentCore Gateway

In Section 5 you built MCP servers and clients from scratch, and saw how an agent discovers and calls tools without custom integration code. That works well with a handful of servers whose tool code you write yourself.

Most enterprises don't have MCP servers. They have hundreds of REST APIs described by OpenAPI specs, internal services behind authentication layers, and legacy systems nobody wants to touch. Writing an MCP wrapper for each one, with auth, error handling, retries and rate limiting, is a full-time job. And when the API team changes an endpoint or rotates credentials, you're back in the code.

AgentCore Gateway bridges that gap. Point it at an OpenAPI spec and it generates MCP tools automatically: no server code to write, no wrapper to maintain. Your agent discovers those tools exactly the way it discovered tools from your Section 5 servers, except now they come from your existing API catalogue.



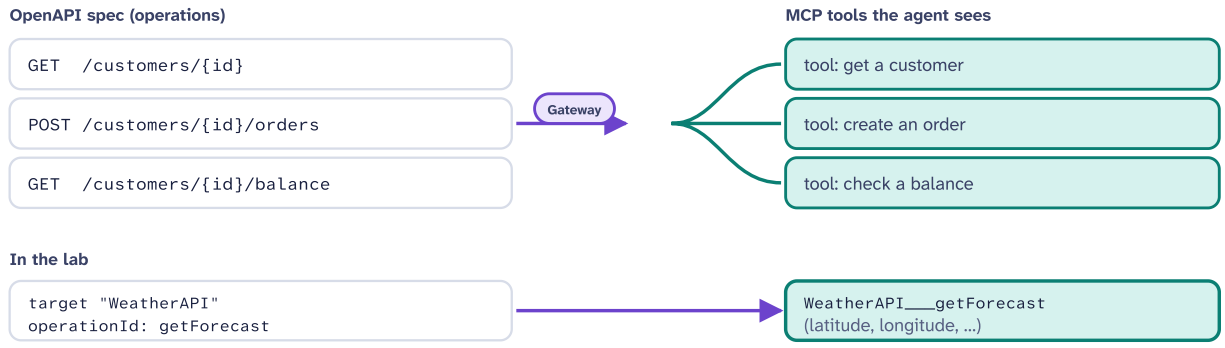

**One front door, many backends.** The agent sees a single list of MCP tools; the gateway knows which backend each one calls.

Gateway supports several backend types: an OpenAPI spec for existing REST APIs, a Lambda function for custom logic, or a remote MCP server for tools that already speak the protocol. You can combine them into a single catalogue. The agent doesn't need to know that *check inventory* hits a Lambda function, *send notification* goes through an API Gateway endpoint, and *update CRM* talks to Salesforce. To the agent they're all just tools.

> ✦ **Think of it like this**
>
> Gateway is a universal travel adapter for your company's APIs. Each API has its own plug shape (REST, Lambda, MCP), but the agent only has one socket: MCP. Gateway converts every plug to that socket, and it also carries the right key for each door it opens.

## Targets

Gateway calls each registered API a target. You register a target by pointing the gateway at an API definition: an OpenAPI spec, a Smithy model, a Lambda function, an API Gateway stage, or a remote MCP server. Gateway reads it, extracts each operation and generates an MCP tool for each, with descriptions, parameter schemas and return types taken straight from the definition.




**Every operation becomes a tool.** Tool names follow `<TargetName>___<operationId>` (three underscores), so tools from different targets never clash.

Here's how the lab creates a gateway and registers a REST API whose spec is stored in S3. First the gateway itself, the front door:

agentcore-gateway notebook · create the gateway Copy

```
def wait_for(get_fn, ready=("READY",), failed=("FAILED", "CREATE_UNSUC
    while True:
        status = get_fn()["status"]
        print(f"  {label} status: {status}")
        if status in ready:
            return
        if status in failed:
            raise RuntimeError(f"{label} failed: {get_fn()}")
        time.sleep(5)


gateway = control.create_gateway(
    name=GATEWAY_NAME,
    roleArn=role_arn,
    protocolType="MCP",
    authorizerType="NONE",
    description="Demo gateway that exposes a REST API as MCP tools",
)
GATEWAY_ID = gateway["gatewayId"]
wait_for(lambda: control.get_gateway(gatewayIdentifier=GATEWAY_ID), la
```

```
GATEWAY_URL = control.get_gateway(gatewayIdentifier=GATEWAY_ID)["gatew
print("Gateway URL:", GATEWAY_URL)
```

| Argument | Meaning |
|---|---|
| `roleArn` | The IAM role the gateway assumes when it works on the agent's behalf (read its config, fetch credentials, read the spec) |
| `protocolType="MCP"` | The gateway speaks MCP to agents |
| `authorizerType` | Who may connect. `CUSTOM_JWT` : OAuth tokens from Cognito, Okta, Entra ID or Auth0, for user-facing apps. `AWS_IAM` : callers sign requests with IAM credentials, for service-to-service calls inside AWS (no separate OAuth provider needed). `NONE` : anyone with the URL, for short-lived demos only. |

> ! **The lab uses `NONE` on purpose, and deletes the gateway at the end**
>
> It keeps the client code simple in class. The production pattern is `authorizerType="AWS_IAM"` or `"CUSTOM_JWT"` .

Then register the API as a target. You're telling the gateway: "here's an OpenAPI spec in S3: read it and turn every endpoint into an MCP tool."

agentcore-gateway notebook · register the REST API as a target Copy

```
target = control.create_gateway_target(
    gatewayIdentifier=GATEWAY_ID,
    name=TARGET_NAME,
    description="Open-Meteo weather forecast API",
    targetConfiguration={
        "mcp": {
            "openApiSchema": {
                "s3": {"uri": SPEC_URI}
            }
        }
    },
    credentialProviderConfigurations=[{
```

```
            "credentialProviderType": "API_KEY",
            "credentialProvider": {
                "apiKeyCredentialProvider": {
                    "providerArn": API_KEY_PROVIDER_ARN,
                    "credentialParameterName": "X-Api-Key",
                    "credentialLocation": "HEADER",
                }
            },
        }],
    )
    TARGET_ID = target["targetId"]
    wait_for(lambda: control.get_gateway_target(gatewayIdentifier=GATEWAY_
```

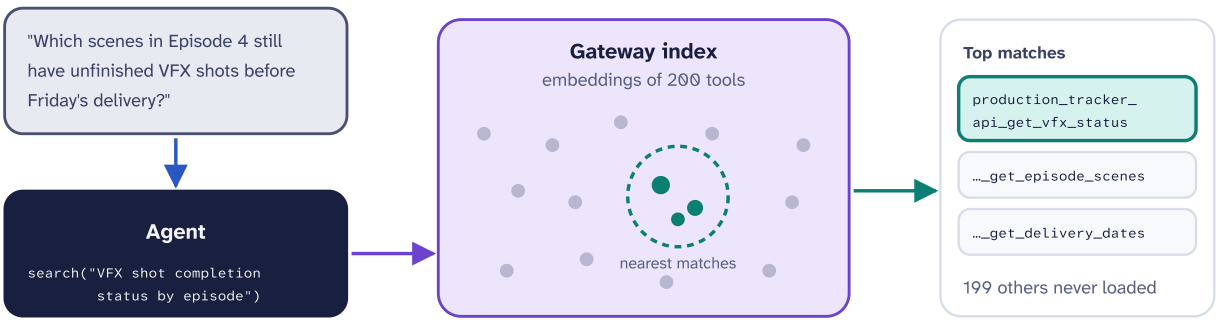
`targetConfiguration` points at the spec. `credentialProviderConfigurations` tells the gateway how to authenticate to *this* API: fetch an API key from the credential provider and send it in the `X-Api-Key` header on every call. The agent never sees that key.

Where this really pays off is composition. Register your customer API, a Lambda function that runs fraud checks and a third-party weather service as targets on the same gateway, and your agent sees one unified tool catalogue on one MCP connection.

## Semantic search: when there are too many tools

With five or ten tools you can load every definition into the agent's prompt and let the model pick; that's what you did in Section 5. At 200 tools you have a problem: the definitions alone eat a big chunk of the context window, and models get noticeably worse at choosing the right tool as the list grows.

Gateway handles this with semantic search. When you register targets, it creates vector embeddings of each tool's name, description and parameters and indexes them; the index updates itself as targets change. The search is exposed as one more MCP tool, `x_amz_bedrock_agentcore_search`. You switch it on when you create the gateway, by adding `protocolConfiguration={"mcp": {"searchType": "SEMANTIC"}}` to `create_gateway`. (The lab gateway has one tool and doesn't switch it on, so its tool list shows only `WeatherAPI___getForecast`.)




**Search first, then call.** The agent loads a short list instead of 200 definitions: fewer tokens per request and better tool choices.

! **Search is only as good as your API documentation**

If your OpenAPI spec has well-named operations with clear descriptions, the embeddings are useful and the matches accurate. Operations called `op_17` with no description will be hard for anyone to find, including the search.

Try it **200 tools: load them all, or search?**

**The user asks**

**Which scenes in Episode 4 still have unfinished VFX shots before Friday's delivery?**

☐ **The API spec is poorly documented (vague names, no descriptions)**

⌕ search("VFX shot completion status by episode") 1 call

| | | |
|---|---|---|
| 1 | production_tracker_api_get_vfx_status | similarity 0.91 |
| 2 | production_tracker_api_get_episode_scenes | similarity 0.78 |
| 3 | delivery_api_get_delivery_schedule | similarity 0.64 |

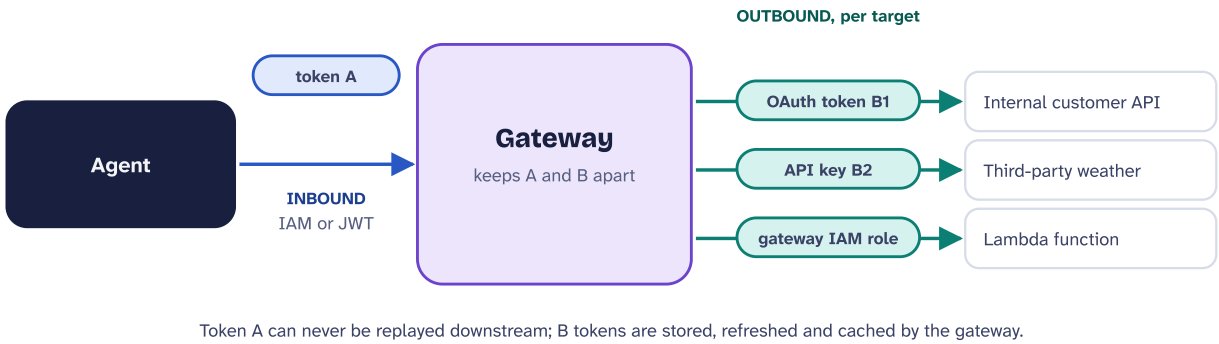
TOOL DEFINITIONS IN THE PROMPT
**3 + search**

ROUGH PROMPT TOKENS FOR TOOLS
**≈ 600**

**The right tool comes first.** The agent loads three definitions instead of 200 (about 600 tokens instead of 30,000) and calls `production_tracker_api_get_vfx_status` directly. Token counts are rough illustrations.

## Authentication: inbound and outbound

Any production deployment must answer two questions: **who is allowed to call the gateway**, and **how does the gateway prove its identity to the APIs it calls on your behalf?**




**Two different keys for two different doors.** Auditors in regulated industries check exactly this separation.

**Inbound** (who can reach the gateway): pick IAM or JWT. If your agent already runs on AWS with an IAM role (on Runtime, Lambda or ECS), IAM is the simpler path: grant that role permission to invoke the gateway and you're done. When end-user identity needs to flow through, use JWTs from Cognito, Okta or any OAuth 2.0 provider.

**Outbound** (how the gateway authenticates to each API) is configured per target: an OAuth token for the internal customer API, an API key for a third-party weather service, the gateway's own IAM role for a Lambda function. (OpenAPI targets always expect an OAuth or API-key credential, which is why the lab registers a placeholder key for a free API.) The credentials are kept in secure storage (backed by AWS Secrets Manager), tokens are refreshed and cached so the gateway doesn't hammer downstream identity providers.

The two sets of tokens are completely separate, with different scopes. A downstream API can't take the inbound token and reuse it to impersonate your agent elsewhere. Gateway handles the mechanics of authenticating to downstream APIs, but a bigger question remains: *who is the user behind the request, and what are they allowed to do?* That's the job of AgentCore Identity (6.9).

## Lab: turn an existing REST API into agent tools

The notebook `agentcore-gateway/gateway_openapi_target.ipynb` turns a real REST API (a subset of the free Open-Meteo forecast API) into an MCP tool with zero tool code, then lets a Strands agent use it.

```
the lab in one picture (from the notebook)

Strands agent ──MCP──► AgentCore Gateway ──HTTPS──► REST API (Open-
Meteo weather)
                            │
                            └─ reads the OpenAPI spec from S3 and
creates the tools
```

You need AWS credentials with permissions for `bedrock-agentcore:*` (gateway and identity), creating and deleting IAM roles, S3 on the demo bucket, and Secrets Manager (used behind the scenes to store the key).

## The spec

agentcore-gateway/weather_openapi.json Copy

```
{
  "openapi": "3.0.3",
  "info": {
    "title": "Weather Forecast API",
    "version": "1.0.0",
    "description": "Subset of the free Open-Meteo forecast API (https:
  },
  "servers": [{ "url": "https://api.open-meteo.com" }],
  "paths": {
    "/v1/forecast": {
      "get": {
        "operationId": "getForecast",
        "summary": "Get current weather and a daily forecast for a lat
        "description": "Returns current temperature and wind, plus dai
        "parameters": [
          { "name": "latitude", "in": "query", "required": true, "desc
          { "name": "longitude", "in": "query", "required": true, "des
          { "name": "current", "in": "query", "required": false, "desc
          { "name": "daily", "in": "query", "required": false, "descri
          { "name": "timezone", "in": "query", "required": false, "des
          { "name": "forecast_days", "in": "query", "required": false,
        ],
        "responses": {
          "200": {
            "description": "Forecast data",
            "content": { "application/json": { "schema": { "type": "ob
```

```
            }
          }
        }
      }
    }
  }
```

One operation, `getForecast`, with a clear summary, a description and documented parameters (even an example latitude for Bangalore). That documentation is exactly what the agent reads when deciding how to call the tool.

### Setup and Step 1: upload the spec to S3

notebook · names and clients Copy

```
import json
import os
import time
import uuid

import boto3

MODEL_ID = os.getenv("BEDROCK_MODEL_ID", "global.anthropic.claude-sonn
REGION = os.getenv("AWS_REGION", os.getenv("AWS_DEFAULT_REGION", "us-e

session = boto3.Session(region_name=REGION)
account_id = session.client("sts").get_caller_identity()["Account"]
s3 = session.client("s3")
iam = session.client("iam")
control = session.client("bedrock-agentcore-control")

suffix = uuid.uuid4().hex[:6]
GATEWAY_NAME = f"weather-gateway-{suffix}"
TARGET_NAME = "WeatherAPI"
BUCKET = f"gateway-specs-{account_id}-{REGION}"
SPEC_KEY = "weather_openapi.json"
ROLE_NAME = f"AgentCoreGatewayDemoRole-{suffix}"
API_KEY_PROVIDER = f"weather-api-key-{suffix}"

print(f"Account {account_id} | Region {REGION} | Gateway {GATEWAY_NAME
```

notebook · step 1 Copy

```
try:
    if REGION == "us-east-1":
        s3.create_bucket(Bucket=BUCKET)
    else:
        s3.create_bucket(Bucket=BUCKET, CreateBucketConfiguration={"Lo
except s3.exceptions.BucketAlreadyOwnedByYou:
    pass

s3.upload_file("weather_openapi.json", BUCKET, SPEC_KEY)
SPEC_URI = f"s3://{BUCKET}/{SPEC_KEY}"
print("Spec uploaded:", SPEC_URI)
```

### Step 2: store the API credential in AgentCore Identity

notebook · step 2 Copy

```
provider = control.create_api_key_credential_provider(
    name=API_KEY_PROVIDER,
    apiKey="demo-key-open-meteo-does-not-need-one",
)
API_KEY_PROVIDER_ARN = provider["credentialProviderArn"]
print("Credential provider:", API_KEY_PROVIDER_ARN)
```

OpenAPI targets need an outbound credential that the gateway attaches to every call. Real APIs use a real key or OAuth; Open-Meteo is free and ignores extra headers, so a placeholder works here. The key is stored in AgentCore Identity, never in your code or the spec.

### Step 3: an IAM role for the gateway

▶ **See the role and its permissions**

The **trust policy** lets only the AgentCore service, acting for your account, assume the role. The **permissions** are least privilege: read its own gateway config, fetch the API

key from the token vault, read the secret, and read objects in the one demo bucket. Nothing else.

### Steps 4 and 5: create the gateway and the target

These are the two cells you saw above. The target is named `WeatherAPI`, so the gateway creates the tool `WeatherAPI___getForecast`.

### Step 6: connect a Strands agent

notebook · step 6 Copy

```
from mcp.client.streamable_http import streamablehttp_client
from strands import Agent
from strands.models.bedrock import BedrockModel
from strands.tools.mcp import MCPClient

gateway_client = MCPClient(lambda: streamablehttp_client(GATEWAY_URL))

with gateway_client:
    tools = gateway_client.list_tools_sync()
    print("Tools exposed by the gateway:", [t.tool_name for t in tools

    agent = Agent(
        model=BedrockModel(model_id=MODEL_ID, region_name=REGION),
        tools=tools,
        system_prompt="You are a travel assistant. Use the weather too
    )
    agent("I'm visiting Bangalore and Mumbai this week. What's the for
```

The gateway is just an MCP server over **Streamable HTTP**, so this is the same `MCPClient` pattern as Section 5, with `streamablehttp_client(GATEWAY_URL)` instead of a local process.

output illustrative; forecasts and wording vary

```
Tools exposed by the gateway: ['WeatherAPI___getForecast']

Tool #1: WeatherAPI___getForecast
Tool #2: WeatherAPI___getForecast
Here's your 3-day forecast:
```

```
**Bangalore** (12.97, 77.59): 27-28 °C highs, 19-20 °C lows, light
showers on day 2.
**Mumbai** (19.08, 72.88): around 32 °C highs, 26 °C lows, mostly dry.

Pack a light umbrella for Bangalore!
```

Notice the model worked out latitude and longitude for each city on its own: the spec told it those are the required parameters.

### Step 7: clean up

notebook · step 7 Copy

```
control.delete_gateway_target(gatewayIdentifier=GATEWAY_ID, targetId=T
while control.list_gateway_targets(gatewayIdentifier=GATEWAY_ID).get("
    time.sleep(3)
control.delete_gateway(gatewayIdentifier=GATEWAY_ID)
print("Deleted gateway", GATEWAY_ID)

control.delete_api_key_credential_provider(name=API_KEY_PROVIDER)
print("Deleted credential provider", API_KEY_PROVIDER)

iam.delete_role_policy(RoleName=ROLE_NAME, PolicyName="GatewayPermissi
iam.delete_role(RoleName=ROLE_NAME)
print("Deleted role", ROLE_NAME)

s3.delete_object(Bucket=BUCKET, Key=SPEC_KEY)
s3.delete_bucket(Bucket=BUCKET)
print("Deleted bucket", BUCKET)
```

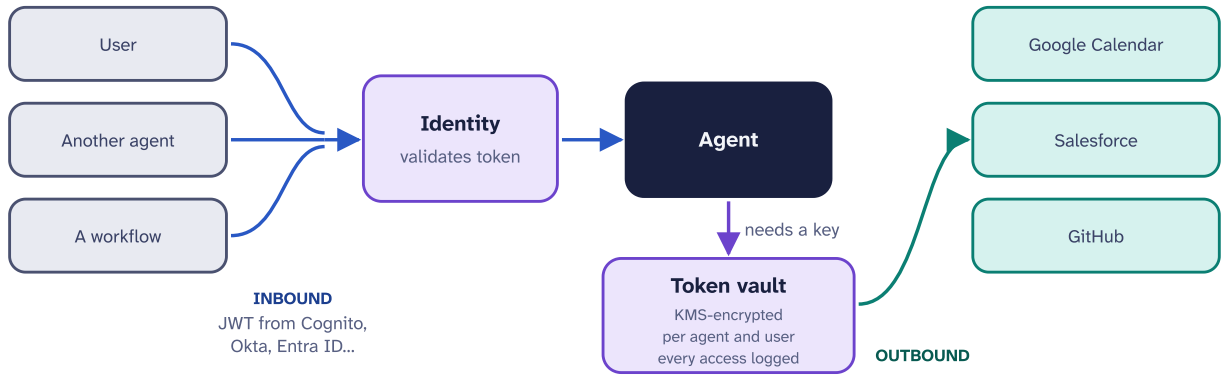
→ **Use your own API**

Replace `weather_openapi.json` with your spec and put your real API key (or an OAuth provider) in Step 2. The same gateway can also expose Lambda functions, Smithy models and other MCP servers, and in Section 7 it enforces Cedar policies on every tool call.

# 6.9 AgentCore Identity

Every production agent faces a two-sided authentication problem.

On one side, **callers** must prove who they are before the agent will talk to them, whether the caller is a user, another agent or an automated workflow. On the other side, when the agent calls an **external service** (a CRM, a calendar API, a code repository) it needs credentials for that service. Without the first, anyone can hit your agent endpoint. Without the second, you end up embedding secrets in environment variables or hard-coding API keys, which is how breaches happen.




**Who's calling, and on whose behalf the agent acts.** Identity keeps those two facts connected.

**Inbound:** Identity supports JWT-based authorisation. Your identity provider (Amazon Cognito, Okta, Microsoft Entra ID, or any standard OAuth 2.0 / OIDC provider) issues the token, and Identity validates it before the request reaches your agent.

**Outbound:** Identity manages the credentials your agent needs for external services, storing OAuth 2.0 tokens and API keys in a secure token vault rather than in your code or configuration. Credentials are encrypted with AWS KMS, bound to specific agent-user combinations, and every access is logged for audit.

## Two kinds of outbound access




**Many agents need both at once:** user-delegated access for personal data and machine-to-machine access for shared resources.

In your agent code this shows up as decorators. Put `@requires_access_token` or `@requires_api_key` on a function and the SDK fetches, injects and refreshes the credential for you. Identity has built-in credential providers for services like Google, GitHub, Slack, Salesforce and Atlassian, with endpoints and provider-specific settings pre-configured; for anything else you can set up a custom OAuth 2.0 provider.

pattern sketch with the bedrock-agentcore SDK — illustrative

```
from bedrock_agentcore.identity.auth import requires_access_token, req

# On behalf of the user: OAuth authorisation code flow ("USER_FEDERATI
@requires_access_token(
    provider_name="google-calendar",          # a credential provider
    scopes=["https://www.googleapis.com/auth/calendar.readonly"],
    auth_flow="USER_FEDERATION",
    on_auth_url=lambda url: print("Ask the user to open:", url),
)
def list_my_events(*, access_token: str):
    ...  # call the Calendar API with access_token

# As itself: OAuth client credentials ("M2M")
@requires_access_token(
    provider_name="warehouse-oauth",
    scopes=["warehouse/read"],
    auth_flow="M2M",
```

```
)
def query_warehouse(sql: str, *, access_token: str):
    ...  # call the data warehouse with access_token

# A static API key from the token vault
@requires_api_key(provider_name="weather-api-key")
def get_forecast(city: str, *, api_key: str):
    ...  # call the weather API with api_key
```

Because Identity validates the inbound token *and* manages outbound credentials, it keeps the link between who is calling and what the agent is doing on their behalf. That context, the caller's identity paired with the agent's identity, is what lets the rest of AgentCore enforce per-user, per-action governance.

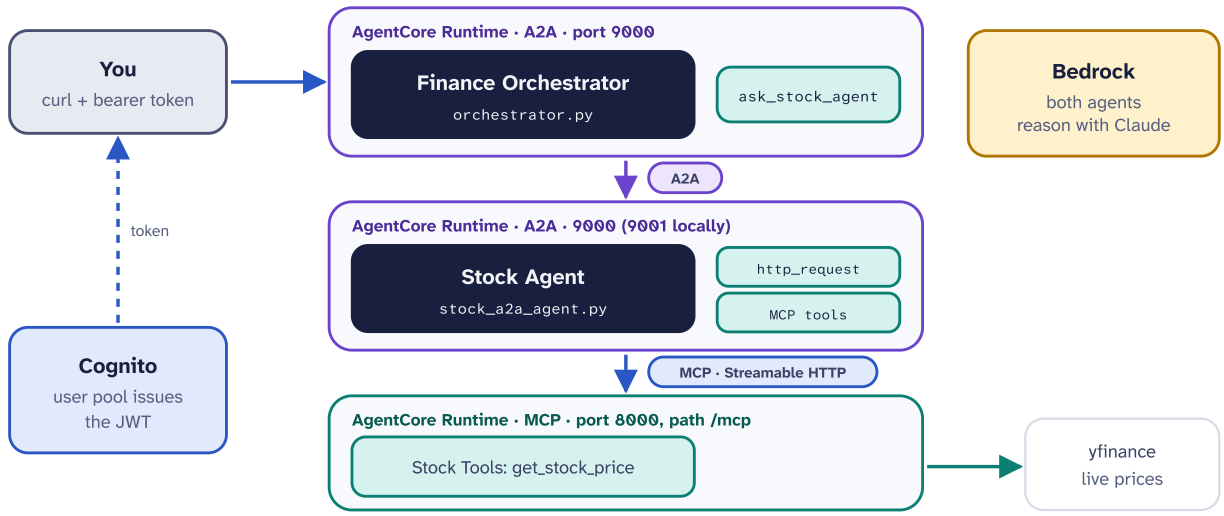
> </> **You've already used it**
>
> Step 2 of the Gateway lab created an **API key credential provider** in Identity, and the gateway pulled the key from the token vault on every call. The multi-agent lab next uses a **Cognito JWT** for inbound authentication.

> i **On Lambda or ECS?**
>
> You build credential management yourself: store tokens in Secrets Manager, implement OAuth flows in your app, handle token refresh, manage the security. It works, but it's undifferentiated work that Identity provides as a managed service.

---

## 6.10 Lab: MCP + A2A on AgentCore

In Section 5 you ran MCP and A2A on your laptop. This lab puts both protocols into production: a finance orchestrator talks A2A to a stock agent, which gets live prices from an MCP server. All three run on AgentCore Runtime, protected by OAuth.




**A2A between the agents, MCP to the tools, OAuth at every door.** Three separate runtimes, each in its own microVMs, each scaling on its own.

| File | Protocol | Local port | What it does |
|---|---|---|---|
| `stock_mcp_server.py` | MCP | 8000 | `get_stock_price` tool backed by yfinance |
| `stock_a2a_agent.py` | A2A | 9001 | Agent with the MCP tools + `http_request` |
| `orchestrator.py` | A2A | 9000 | Agent that calls the Stock Agent through an `ask_stock_agent` tool |

**Ports AgentCore expects**

On AgentCore Runtime, HTTP agents listen on **8080**, MCP servers on **8000** (path `/mcp`), and A2A agents on **9000**. Locally the stock agent uses 9001 so it doesn't clash with the orchestrator; inside the AgentCore container it switches to 9000.

## The MCP server

multi-agent-on-agentcore/stock_mcp_server.py Copy

```python
import yfinance as yf
from mcp.server.fastmcp import FastMCP

# stateless_http=True: every request is independent — required on Agen
mcp = FastMCP("StockTools", host="0.0.0.0", stateless_http=True)


@mcp.tool()
def get_stock_price(ticker: str) -> str:
    """Get the latest stock price for a given ticker symbol (e.g. AAPL
    try:
        stock = yf.Ticker(ticker.strip().upper())
        info = stock.info
        price = info.get("currentPrice") or info.get("regularMarketPri
        name = info.get("shortName", ticker.upper())
        currency = info.get("currency", "USD")
        change = info.get("regularMarketChangePercent")

        if price is None:
            return f"Could not find price for '{ticker}'. Check the ti

        result = f"{name} ({ticker.upper()}): {currency} {price:.2f}"
        if change is not None:
            direction = "▲" if change >= 0 else "▼"
            result += f" {direction} {abs(change):.2f}%"
        return result
    except Exception as e:
        return f"Error fetching {ticker}: {e}"


if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

A FastMCP server like Section 5's, with two production details. It runs over **Streamable HTTP** instead of stdio, because on AgentCore it's a remote service, not a child process (FastMCP's defaults, port `8000` and path `/mcp`, are exactly what AgentCore expects). And `stateless_http=True` makes every request independent, which AgentCore requires: the platform itself manages the MCP session (it adds an `Mcp-Session-Id` header and routes on it), so the server mustn't keep its own per-connection state.

## The Stock Agent (A2A server with MCP tools)

multi-agent-on-agentcore/stock_a2a_agent.py · settings from the environment

Copy

```
logging.basicConfig(level=logging.INFO)

MODEL_ID = os.getenv("BEDROCK_MODEL_ID", "global.anthropic.claude-sonn
STOCK_MCP_URL = os.environ.get("STOCK_MCP_URL", "http://localhost:8000
# Set automatically on AgentCore Runtime; used as the public URL in th
RUNTIME_URL = os.environ.get("AGENTCORE_RUNTIME_URL", "http://127.0.0.
# Bearer token for calling the MCP server when it is deployed behind C
BEARER_TOKEN = os.environ.get("AGENT_BEARER_TOKEN")

# Locally this agent uses 9001 (the orchestrator has 9000). On AgentCo
# A2A agents must listen on 9000 — the deploy command passes PORT=9000
PORT = int(os.environ.get("PORT", "9000" if os.environ.get("DOCKER_CON

MCP_HEADERS = {"Authorization": f"Bearer {BEARER_TOKEN}"} if BEARER_TO
```

Everything that differs between your laptop and AgentCore comes from environment variables: where the MCP server is, the agent's public URL, the bearer token for calling the protected MCP server, and the port. Same code, both places.

multi-agent-on-agentcore/stock_a2a_agent.py · MCP tools + agent

Copy

```
# MCP client for the stock price tools. Passing it straight into tools
# lets the agent manage the connection for us.
mcp_client = MCPClient(lambda: streamablehttp_client(STOCK_MCP_URL, he

stock_agent = Agent(
    name="Stock Agent",
    description="Provides stock prices and financial information.",
    model=BedrockModel(model_id=MODEL_ID),
    system_prompt=(
        "You are a financial assistant. Use get_stock_price to look up
        "Use http_request to fetch data from financial websites when n
        "Always provide the ticker symbol, current price, and any rele
    ),
    tools=[mcp_client, http_request],
```

```
    callback_handler=None,
)
```

The `MCPClient` passes the bearer token as an HTTP header, and it goes straight into `tools=[...]` next to `http_request` (from `strands_tools`), so the agent opens and closes the MCP connection itself.

**multi-agent-on-agentcore/stock_a2a_agent.py · serve over A2A, with a /ping** Copy

```
a2a_server = A2AServer(
    agent=stock_agent,
    http_url=RUNTIME_URL,
    serve_at_root=True,
    skills=SKILLS,
    version="1.0.0",
    enable_a2a_compliant_streaming=True,
)

app = FastAPI()


@app.get("/ping")
def ping():
    return {"status": "healthy"}


app.mount("/", a2a_server.to_fastapi_app())

if __name__ == "__main__":
    print(f"Stock A2A Agent on http://0.0.0.0:{PORT} (MCP: {STOCK_MCP_
    uvicorn.run(app, host="0.0.0.0", port=PORT)
```

1 **`http_url=RUNTIME_URL`**

The URL written into the agent card. A2A clients send messages to the URL in the card, so on AgentCore it must be the agent's public Runtime URL, not `127.0.0.1`.

**2** `serve_at_root=True` + `to_fastapi_app()`

Turns the A2A server into a FastAPI app mounted at `/`, inside a bigger FastAPI app that also has the `/ping` health check AgentCore calls.

**3** `uvicorn.run(..., host="0.0.0.0", port=PORT)`

Listen on all interfaces (required inside a container) on 9001 locally or 9000 on AgentCore.

▶ **See the skills in the agent card**

## The Finance Orchestrator (A2A client inside a tool)

`multi-agent-on-agentcore/orchestrator.py · the remote agent as a tool` Copy

```
def remote_stock_agent() -> A2AAgent:
    """A client for the remote Stock Agent (adds the bearer token when
    if BEARER_TOKEN:
        http = httpx.AsyncClient(headers={"Authorization": f"Bearer {B
        return A2AAgent(STOCK_A2A_URL, client_config=ClientConfig(http
    return A2AAgent(STOCK_A2A_URL, timeout=300)


@tool
def ask_stock_agent(question: str) -> str:
    """Ask the Stock Agent (a remote A2A agent) for current stock pric

    Args:
        question: A natural-language question, e.g. "What is the curre
    """
    return str(remote_stock_agent()(question))


orchestrator = Agent(
    name="Finance Orchestrator",
    description="Provides financial advice by coordinating with the St
    model=BedrockModel(model_id=MODEL_ID),
```

```
        system_prompt=(
            "You are a financial advisor assistant. When users ask about s
            "investments, or market conditions, use ask_stock_agent to get
            "Provide helpful analysis and context with the data. This is n
        ),
        tools=[ask_stock_agent],
        callback_handler=None,
    )
```

This is the "give the orchestrator an LLM" extension from Section 5's lab, done properly. Strands' `A2AAgent` is a client for a remote A2A agent: call it like a local agent and it fetches the card, sends the message and returns the answer. Wrapping it in an `@tool` lets the orchestrator's model decide *when* to ask the stock agent. When deployed, an `httpx` client adds the bearer token to every A2A request.

▶ **See the rest of `orchestrator.py`** (imports, settings, skills, server)

## Part 1: run everything locally

terminal Copy

```
cd multi-agent-on-agentcore
pip install -r requirements.txt

python stock_mcp_server.py      # Terminal 1
python stock_a2a_agent.py       # Terminal 2
python orchestrator.py          # Terminal 3

# Terminal 4: talk to the orchestrator with a raw A2A JSON-RPC request
curl -s -X POST http://localhost:9000/ \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0", "id": "1", "method": "message/send",
    "params": {"message": {"role": "user",
      "parts": [{"kind": "text", "text": "How is NVDA stock doing toda
      "messageId": "test-1"}}
  }' | jq -r '.result.artifacts[0].parts[0].text'
```

That's the same `message/send` you saw on the wire in Section 5; `jq` digs the answer out of `result.artifacts[0].parts[0].text`. Change the port to 9001 to talk to the stock agent directly.

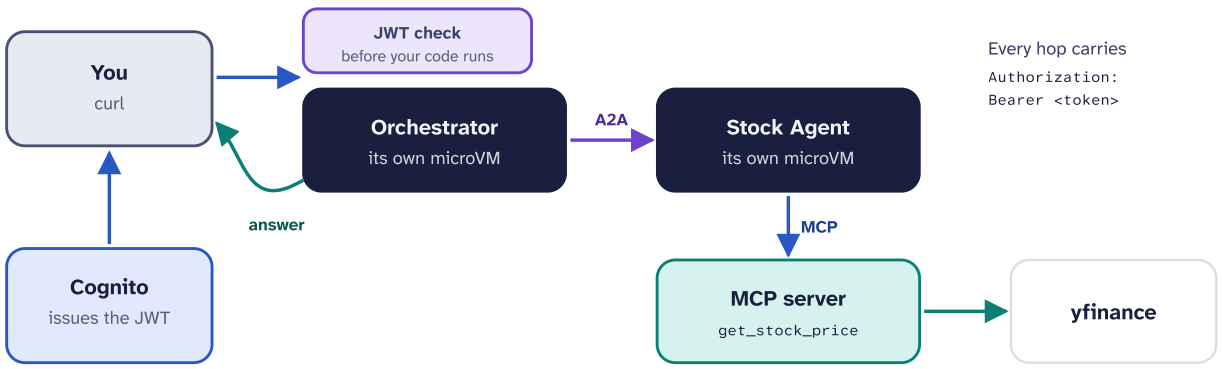
answer

illustrative; prices are live and the wording varies

```
NVIDIA (NVDA) is trading at USD 1xx.xx, ▲ 1.85% today. The move
follows …
Context: semiconductor stocks are up broadly this week. This is not
financial advice.
```

## Part 2: deploy all three to AgentCore Runtime

You need the starter toolkit CLI ( `pip install bedrock-agentcore-starter-toolkit` ; AWS also ships a separate Node.js `agentcore` CLI with different commands, and this lab uses the Python one) and `jq`. Follow the request path, then the commands.




1 Get a token | 2 Call + JWT check | 3 A2A hop | 4 MCP hop | 5 Fetch price | 6 Answer back

Click step 1.

**Three runtimes, one request.** AgentCore checks the token at every door before any of your code runs.

## Step 1: a Cognito user pool for OAuth

terminal · from the multi-agent-on-agentcore folder Copy

```
pip install bedrock-agentcore-starter-toolkit
export REGION=us-east-1
export COGNITO_USERNAME=testuser
export COGNITO_PASSWORD='YourPassword123!'
source ../setup_cognito.sh

export AUTH_CONFIG="{\"customJWTAuthorizer\":{\"discoveryUrl\":\"$DISC
```

setup_cognito.sh · what it creates Copy

```
_cognito_setup() {
  if [ -z "$REGION" ] || [ -z "$COGNITO_USERNAME" ] || [ -z "$COGNITO_
    echo "Error: set REGION, COGNITO_USERNAME and COGNITO_PASSWORD fir
    return 1
  fi

  # User pool
  POOL_ID=$(aws cognito-idp create-user-pool \
    --pool-name "AgentCoreUserPool" \
    --policies '{"PasswordPolicy":{"MinimumLength":8}}' \
    --region "$REGION" --query 'UserPool.Id' --output text)
  [ -n "$POOL_ID" ] && [ "$POOL_ID" != "None" ] || { echo "Error: coul

  # App client (public client, username/password auth)
  CLIENT_ID=$(aws cognito-idp create-user-pool-client \
    --user-pool-id "$POOL_ID" \
    --client-name "AgentCoreClient" \
    --no-generate-secret \
    --explicit-auth-flows "ALLOW_USER_PASSWORD_AUTH" "ALLOW_REFRESH_TO
    --region "$REGION" --query 'UserPoolClient.ClientId' --output text
  [ -n "$CLIENT_ID" ] && [ "$CLIENT_ID" != "None" ] || { echo "Error:

  # User with a permanent password
  aws cognito-idp admin-create-user \
    --user-pool-id "$POOL_ID" --username "$COGNITO_USERNAME" \
    --region "$REGION" --message-action SUPPRESS > /dev/null || return
  aws cognito-idp admin-set-user-password \
    --user-pool-id "$POOL_ID" --username "$COGNITO_USERNAME" --passwor
```

```
    --region "$REGION" --permanent > /dev/null || return 1

  DISCOVERY_URL="https://cognito-idp.$REGION.amazonaws.com/$POOL_ID/.w
  export POOL_ID CLIENT_ID DISCOVERY_URL
  cognito_refresh_token
}
```

A Cognito user pool (the user directory), an app client allowed to log in with username and password, and a user with a permanent password. The **discovery URL** is where AgentCore finds Cognito's public keys to verify tokens. `AUTH_CONFIG` tells each runtime: accept only JWTs from this pool, issued to this app client. Note it's `source` d, not run, so the exported variables stay in your shell.

**setup_cognito.sh · getting a token** Copy

```
# Get a fresh access token (they expire after 1 hour). Call this again
cognito_refresh_token() {
  BEARER_TOKEN=$(aws cognito-idp initiate-auth \
    --client-id "$CLIENT_ID" \
    --auth-flow USER_PASSWORD_AUTH \
    --auth-parameters "USERNAME=$COGNITO_USERNAME,PASSWORD=$COGNITO_PA
    --region "$REGION" --query 'AuthenticationResult.AccessToken' --ou
  export BEARER_TOKEN

  echo ""
  echo "Pool ID:       $POOL_ID"
  echo "Client ID:     $CLIENT_ID"
  echo "Discovery URL: $DISCOVERY_URL"
  echo "Bearer Token:  ${BEARER_TOKEN:0:25}...  (exported as \$BEARER_
  echo ""
  echo "Token expires in 1 hour. Refresh with:  cognito_refresh_token"
}
```

> ! **Tokens expire after one hour**
>
> Run `cognito_refresh_token` in the same shell to get a new one. Requests with an expired token are rejected before they reach your agent.

## Step 2: deploy the MCP server

terminal Copy

```
agentcore configure -e stock_mcp_server.py --protocol MCP \
  --requirements-file requirements.txt --authorizer-config "$AUTH_CONF
  --region $REGION --non-interactive
agentcore deploy --agent stock_mcp_server

export MCP_ARN="<stock_mcp_server runtime ARN>"
export STOCK_MCP_URL="https://bedrock-agentcore.$REGION.amazonaws.com/
```

`--protocol MCP` tells Runtime this is an MCP server (port 8000, `/mcp`). Every runtime is reached at a URL built from its ARN; the ARN contains characters like `:` and `/`, so `jq '@uri'` URL-encodes it.

### Step 3: deploy the Stock Agent

terminal Copy

```
agentcore configure -e stock_a2a_agent.py --protocol A2A \
  --requirements-file requirements.txt --authorizer-config "$AUTH_CONF
  --region $REGION --non-interactive
agentcore deploy --agent stock_a2a_agent \
  --env PORT=9000 --env STOCK_MCP_URL=$STOCK_MCP_URL --env AGENT_BEARE

export STOCK_A2A_ARN="<stock_a2a_agent runtime ARN>"
export STOCK_A2A_URL="https://bedrock-agentcore.$REGION.amazonaws.com/

# deploy once more so the agent card advertises its public URL
agentcore deploy --agent stock_a2a_agent --auto-update-on-conflict \
  --env PORT=9000 --env STOCK_MCP_URL=$STOCK_MCP_URL --env AGENT_BEARE
  --env AGENTCORE_RUNTIME_URL=$STOCK_A2A_URL
```

Why deploy twice? You only learn the agent's public URL after the first deploy, but the agent card must contain that URL (A2A clients send their messages to the URL in the card). The second deploy passes it in as `AGENTCORE_RUNTIME_URL`.

### Step 4: deploy the Orchestrator

terminal Copy

```
agentcore configure -e orchestrator.py --protocol A2A \
  --requirements-file requirements.txt --authorizer-config "$AUTH_CONF
  --region $REGION --non-interactive
agentcore deploy --agent orchestrator \
  --env STOCK_A2A_URL=$STOCK_A2A_URL --env AGENT_BEARER_TOKEN=$BEARER_

export ORCH_ARN="<orchestrator runtime ARN>"
```

### Step 5: test

terminal Copy

```
curl -s -X POST "https://bedrock-agentcore.$REGION.amazonaws.com/runti
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $BEARER_TOKEN" \
  -H "X-Amzn-Bedrock-AgentCore-Runtime-Session-Id: $(uuidgen)$(uuidgen
  -d '{
    "jsonrpc": "2.0", "id": "1", "method": "message/send",
    "params": {"message": {"role": "user",
      "parts": [{"kind": "text", "text": "How is NVDA stock doing toda
      "messageId": "test-1"}}
  }' | jq .
```

The `Authorization` header carries your Cognito token. The session-ID header picks the session (and microVM); two UUIDs glued together comfortably meet the minimum length of 33 characters. The body is the same A2A message as the local test.

### Step 6: clean up

terminal Copy

```
agentcore destroy --agent orchestrator --force --delete-ecr-repo
agentcore destroy --agent stock_a2a_agent --force --delete-ecr-repo
agentcore destroy --agent stock_mcp_server --force --delete-ecr-repo
aws cognito-idp delete-user-pool --user-pool-id $POOL_ID --region $REG
```

The runtimes' CloudWatch log groups ( `/aws/bedrock-agentcore/runtimes/…` ) are kept so you can read them later; delete them in the CloudWatch console when you no longer need them.

> ! **Agent-to-agent auth: classroom shortcut vs production**
>
> For the demo, the deployed agents reuse *your* Cognito access token ( `AGENT_BEARER_TOKEN` ) to call each other, and it expires after an hour. Refreshing your token doesn't update the copy inside the deployed agents, so after an hour redeploy steps 3 and 4 with a fresh `$BEARER_TOKEN` . In production, give each agent its own machine identity with AgentCore Identity: the OAuth client-credentials (machine-to-machine) flow from 6.9.

> i **Changing the model**
>
> Set `BEDROCK_MODEL_ID` , for example with `--env BEDROCK_MODEL_ID=…` on deploy.

---

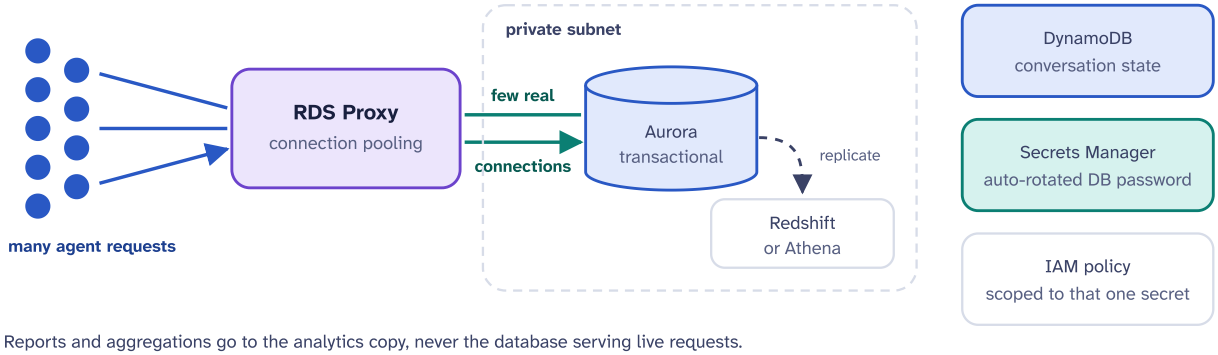
# 6.11 Enterprise integration patterns

Once deployed, your agent has to talk to other systems: databases, internal APIs, third-party services, event streams. If you use AgentCore Gateway, it handles much of this (point it at a spec, get tools with auth and error handling built in). The patterns below apply on Lambda or ECS, or whenever you need behaviour Gateway doesn't cover.

## Databases

Databases are often the first integration to break at scale. An agent serving forty concurrent users, each triggering two or three queries, will exhaust a typical Amazon Aurora (AWS's managed relational database) connection pool within minutes. Amazon RDS Proxy sits between your agent and the database and fixes this by multiplexing hundreds of agent requests over a smaller set of real database

connections. The agent code doesn't change; the infrastructure absorbs the concurrency.




**Pool the connections, separate the workloads, lock the keys away.**

Three more rules: keep analytical queries (reports, aggregations) off your transactional database and replicate to Redshift or Athena instead; keep conversation state in DynamoDB (as in Section 3); and keep credentials in Secrets Manager with automatic rotation, not in environment variables. Scope the IAM policy to the exact secret ARN the agent needs, and keep database traffic inside a private subnet.

Try it **Forty users and a connection pool**

Concurrent users = 40

Queries each user triggers = 3

☐ Put RDS Proxy in front of the database

| CONNECTIONS ASKED FOR | REAL DB CONNECTIONS | DATABASE LIMIT (EXAMPLE) |
|---|---|---|
| 120 | 120 | 100 |

**Too many connections.** 120 connections against a limit of 100: 20 queries fail with "too many connections", and the agent's tools start erroring for real users.

## External APIs

External APIs fail in ways your agent must handle gracefully. Internal services go down during deployments. Third-party providers enforce rate limits and return `429`s. Some endpoints just stop responding, with no error at all.

The rules: set a **timeout** on every call; **retry with exponential backoff**; and return a **structured error the agent can reason about**. *"I couldn't complete the underwriting check; proceeding with a preliminary assessment"* is far better than a raw 500. Circuit breakers stop your agent burning time on timeouts when a service is clearly down. For rate-limited APIs, respect `Retry-After` headers, and cache responses when the data doesn't change between calls.

pattern sketch: a resilient tool — illustrative

```
import random, time
import httpx
from strands import tool

FAILS, OPEN_UNTIL = 0, 0.0          # a tiny circuit breaker
```

```python
@tool
def underwriting_check(applicant_id: str) -> dict:
    """Run the underwriting check for a loan applicant."""
    global FAILS, OPEN_UNTIL
    if time.time() < OPEN_UNTIL:                    # breaker open: fa
        return {"status": "unavailable", "detail": "Underwriting servi
                "proceed with a preliminary assessment and say so."}
    for attempt in range(3):
        try:
            r = httpx.get(f"https://underwriting.internal/check/{appli
            if r.status_code == 429:                # rate limited:
                try:
                    wait = float(r.headers.get("Retry-After", 2))
                except ValueError:                  # Retry-After c
                    wait = 2
                time.sleep(wait)
                continue
            r.raise_for_status()
            FAILS = 0
            return {"status": "ok", "result": r.json()}
        except (httpx.TimeoutException, httpx.HTTPError):
            if attempt < 2:
                time.sleep(2 ** attempt + random.random())  # 1 s, the
    FAILS += 1
    if FAILS >= 2:
        OPEN_UNTIL = time.time() + 30               # stop trying f
    return {"status": "unavailable", "detail": "Underwriting check fai
            "proceed with a preliminary assessment and say so."}
```

**Think of it like this**

A circuit breaker in your house cuts the power when something's wrong, so you don't keep flipping a switch that only makes sparks. A software circuit breaker does the same: after repeated failures it stops calling the broken service for a while and answers "unavailable" instantly, then tries again later.

Try it **When the underwriting API misbehaves**

The API is...

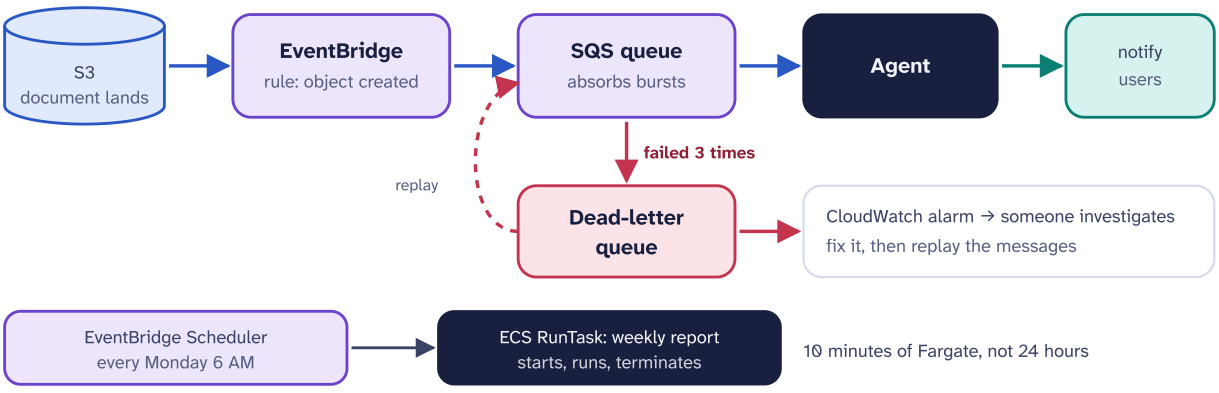
Your tool uses...

| CALLS | TIME THE USER WAITED |
| --- | --- |
| 0 | 0 s |

Pick how the API behaves and what your tool does, then call it a few times.

## Event-driven integration

Event-driven integration decouples your agent from the systems that feed it. A document lands in Amazon S3; an Amazon EventBridge rule picks up the "object created" event and routes it to the agent (you turn on EventBridge notifications for the bucket first). The upstream system doesn't know the agent exists, and the agent doesn't know about the notification system that alerts users when processing finishes.

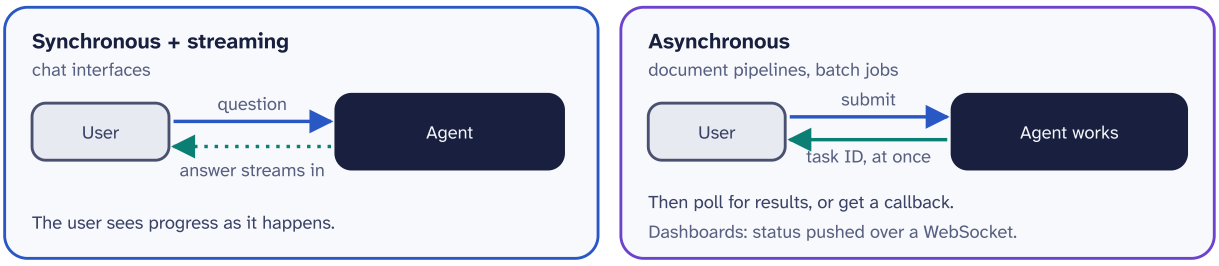



**Nobody calls the agent directly.** Events flow in, failures are parked for humans, and scheduled jobs run only as long as they need.

When volume spikes, put an SQS queue between EventBridge and the agent to absorb bursts. Messages that fail three times go to a dead-letter queue; a CloudWatch alarm

fires, someone investigates, and once the issue is fixed they replay the messages. For scheduled work like weekly reports or batch processing, EventBridge Scheduler starts an ECS **RunTask** instead of a Service: it spins up, runs the job and terminates.

### Synchronous or asynchronous responses?



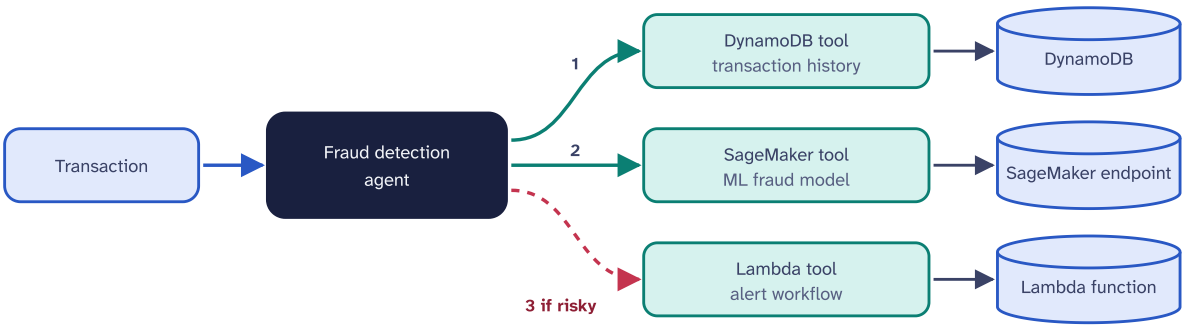

**Most production agents do both.** These are the same synchronous, polling, streaming and push styles you met in A2A in Section 5.

## 6.12 Industry use cases

Here's how these patterns play out in four industries, each with different deployment requirements and integration challenges. Watch for the last line of each: the deployment fit.

### Financial services: fraud detection at transaction speed

A bank's fraud system kept catching suspicious crypto transactions, but it flagged legitimate customers so often that *"Why can't I buy Bitcoin?"* became a daily support ticket. The fraud team was drowning in false positives while real fraudsters slipped through. The fraud model itself, a SageMaker endpoint trained on transaction patterns, was solid. The problem was context: it saw each transaction in isolation and didn't know the customer had been making similar purchases for months.




**Context first, then the model, then escalate only if needed.**

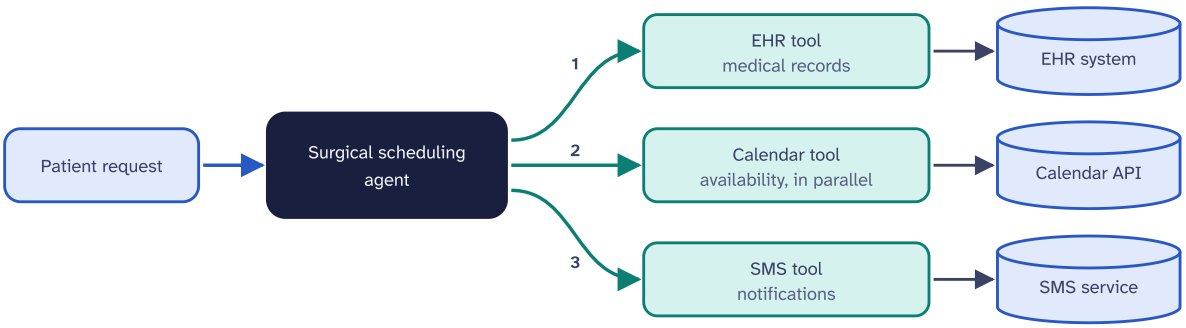
When a transaction arrives, the agent works sequentially: pull the customer's history from DynamoDB, feed that context to the SageMaker model alongside the current transaction, and escalate only when the risk score warrants it; high-risk transactions trigger a Lambda function that starts the investigation workflow. There's no hard-coded `if risk_score > 0.8: call_lambda()` : the agent reads the tool descriptions and orchestrates them based on the situation.

→ **Deployment fit: AWS Lambda**

Each transaction is independent: no multi-turn conversation, no state between invocations. The agent processes one transaction, returns a decision and terminates. It scales to zero in quiet periods and to hundreds of concurrent invocations during market volatility.

## Healthcare: multi-specialty surgical scheduling

Scheduling surgery means coordinating a surgeon, an anaesthesiologist, possibly a specialist, an available operating room, specific equipment that might be in use elsewhere, and a time the patient can actually make. A large hospital system did this manually: coordinators spent hours on the phone, checking calendars and leaving voicemails. By the time they found a slot, the patient had sometimes booked with a competitor.




**Sequential where it must be, parallel where it can be.** Read the requirements first, then check every calendar at once.

When a patient requests surgery, the agent checks the Electronic Health Record first: what kind of surgery, which specialists, which equipment. Then it queries multiple calendars *in parallel* to find a time when everyone is free. Once it finds a slot it books the appointments and sends SMS confirmations. It also checks equipment availability before booking and sends reminders at the right intervals, which it worked out from the tool descriptions and past scheduling patterns. You provide the tools; the model orchestrates them.

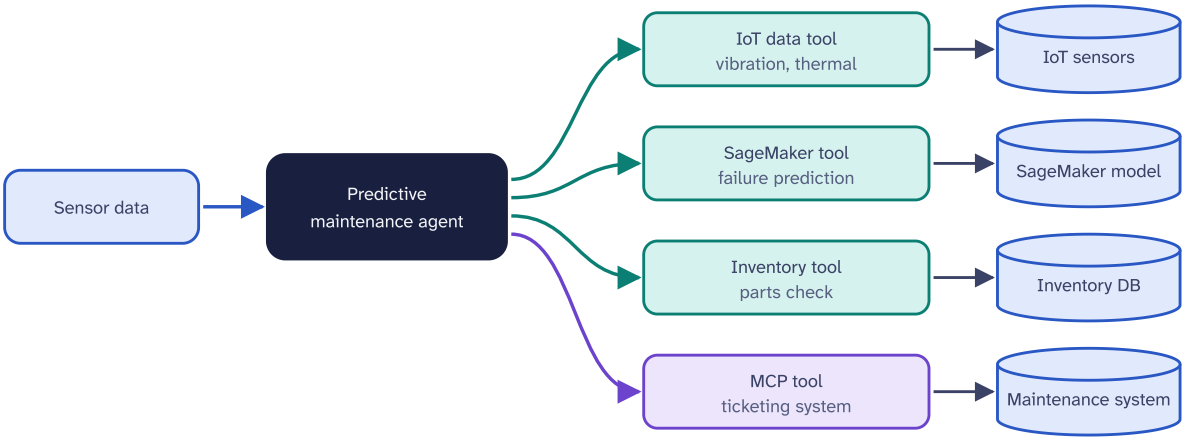
→ **Deployment fit: AgentCore Runtime**

A scheduling conversation might run five messages before a procedure is confirmed, so this is a multi-turn workload. Each coordinator's conversation gets its own isolated session, and the idle timeout matches the natural pace of scheduling (a coordinator might step away to check something with a nurse). Your ECS lab runs this same scenario to teach always-on containers; with many coordinators needing private multi-turn sessions, Runtime is the better production fit.

## Manufacturing: predictive maintenance on the factory floor

A factory had expensive CNC machines with bearings that failed without warning. One day a machine runs fine; the next it's down for three days waiting for parts. Each failure cost roughly $50,000 in lost production. Scheduled maintenance (replace bearings

every six months regardless) was wasteful, and bearings still failed in between. Monitoring vibration sensors by hand produced too much data and too many false alarms.




**A legacy system becomes "just another tool".** The ticketing system was built by another team years ago; an MCP server made it callable without custom integration code.

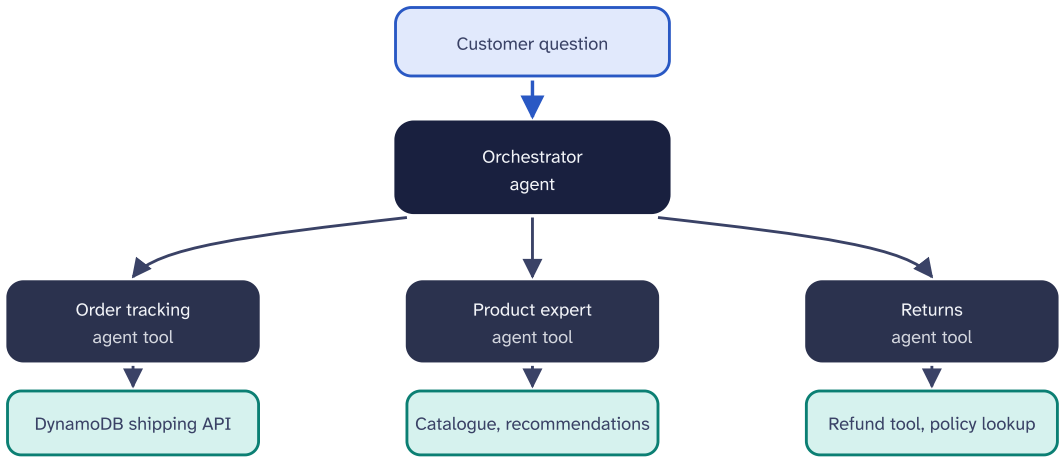
The agent monitors vibration sensors continuously, analyses thermal images for heat patterns, and runs everything through a SageMaker model trained to predict bearing failures. When the risk score crosses a threshold, it checks parts inventory in the warehouse database and creates a maintenance ticket automatically. It also knows to check parts before scheduling maintenance and to schedule during low-production periods, which it learned from the tool descriptions and production schedules, not from hard-coded rules.

→ **Deployment fit: Amazon ECS**

This is continuous monitoring, so it runs on ECS as a long-lived service. Lambda's 15-minute limit rules it out for an agent that never stops processing sensor data, and AgentCore Runtime's per-session isolation doesn't map well to a workload with no distinct user sessions.

## E-commerce: customer support with specialised agents

An online retailer built one agent to handle all customer support: order tracking, product recommendations and returns. It confused return policies with shipping policies, recommended out-of-stock products, and knew a little about everything without knowing enough about anything.




**Agents as tools, from Section 4.** Each specialist knows its own domain deeply; the orchestrator combines their answers.

The fix was three specialised agents (Order Tracking, Product Expert, Returns) wrapped as tools that an orchestrator calls: the agents-as-tools pattern from Section 4. A customer asks about returning an order; the orchestrator calls Order Tracking to check shipment status, then Returns for the policy, and combines both into one reply. Each agent focuses on what it does best, and the orchestrator also uses sentiment analysis to spot frustrated customers who need a human.

→ **Deployment fit: AgentCore Runtime**

Customer support is conversational and multi-turn. Each customer's conversation gets its own isolated session, so there's no cross-customer data leakage even though the agents share tools that touch the same product catalogue and order database.

| Use case | Workload shape | Fit |
|---|---|---|
| **Fraud detection** | One independent decision per event, spiky volume | Lambda |
| **Surgical scheduling** | Multi-turn, one private conversation per coordinator | AgentCore Runtime |
| **Predictive maintenance** | Never stops, no user sessions | ECS |
| **Customer support** | Multi-turn, many customers, shared tools | AgentCore Runtime |

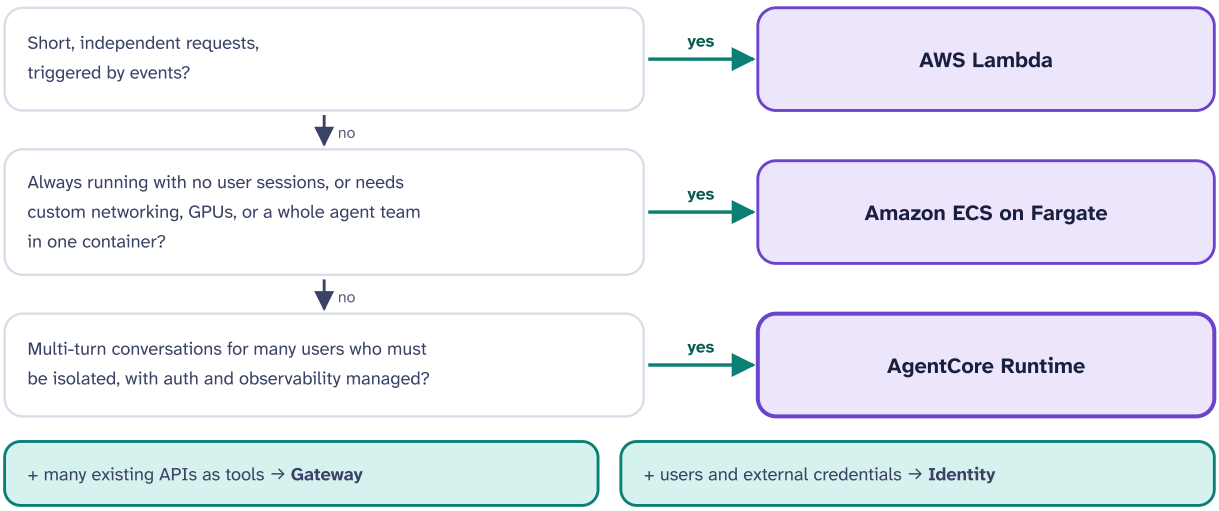
---

## 6.13 Recap and self-check

Your agents are off your laptop and running on AWS.

You saw what changes in production (reliability, security, scale, cost, debugging) and learned to choose a home for each agent: **Lambda** for event-driven, quick tasks; **ECS** for always-on services, continuous workloads and multi-agent teams in one container; **AgentCore** for managed infrastructure with session isolation, auth, policy and observability built in. You met AgentCore's composable architecture (Runtime, Gateway, Identity, Policy, Memory, Observability and Evaluations), each solving one production problem, together or on its own. You deployed agents three ways, turned a REST API into MCP tools, ran MCP and A2A in the cloud behind OAuth, and learned the patterns for connecting agents to databases, APIs and event-driven systems.

But *deployed* and *production-ready* aren't the same thing. Right now, if your agent starts hallucinating, you won't know until a user complains. If a retry loop runs away, you'll find out on the monthly bill. If someone discovers your endpoint, nothing stops them hammering it. That's Section 7.




**The section on one page.** Three homes for the agent, plus two services you can add to any of them.

## You should now be able to explain

| Idea | In one line |
|---|---|
| **What changes in production** | Reliability, security, scale, cost and debugging all stop being optional |
| **Lambda handler** | Expensive, stateless setup at module level; a fresh agent per request; state in DynamoDB if you need it |
| **Cold start** | The extra seconds while a new execution environment starts and runs your setup code |
| **Lambda limits** | 15 minutes per invocation (29 s behind a default REST API), 250 MB zip vs 10 GB image |
| **Bedrock IAM** | Allow both the foundation-model ARNs and the inference-profile ARNs |
| **ECS service, task definition, Fargate** | The kitchen that keeps copies running, the recipe card, and servers you never see |

| Idea | In one line |
|---|---|
| **Browser-carried history** | Simple for demos; server-side sessions survive refreshes and can be audited |
| **AgentCore** | Composable managed services for the agent-specific layer: Runtime, Gateway, Identity, Policy, Memory, Observability, Evaluations |
| **Runtime sessions** | One microVM per session; Active, Idle, Terminated; 15-minute idle timeout; CPU not billed while waiting |
| **`BedrockAgentCoreApp`** | Four lines turn an agent into an HTTP service with `/invocations` and `/ping` |
| **Gateway** | OpenAPI, Lambda, Smithy or MCP targets become MCP tools; semantic search for big catalogues; separate inbound and outbound tokens |
| **Identity** | Validates inbound JWTs; keeps outbound OAuth tokens and API keys in a vault; user-delegated and machine-to-machine flows |
| **Integration patterns** | RDS Proxy, timeouts + backoff + circuit breakers, EventBridge + SQS + DLQ, sync vs async replies |

## When something goes wrong

| What you see | What it usually means and how to fix it |
|---|---|
| **`AccessDeniedException` when the deployed agent calls Bedrock** | The role is missing a Bedrock resource (often the inference-profile ARN or the region-less foundation-model ARN), or the model isn't enabled in your account. Compare with the policies in `template.yaml` and `deploy.sh`. |
| **Model not found / validation error on the model ID** | The ID isn't exact. Copy it from Bedrock and pass it with `BEDROCK_MODEL_ID`. |

| What you see | What it usually means and how to fix it |
| --- | --- |
| **Lambda lab returns `504` after about 29 seconds** | The API Gateway REST API timeout. Send a shorter document, use a faster model, or move to a Function URL or AgentCore Runtime. |
| **The first Lambda request takes 5–15 seconds** | A cold start. Later requests on the same environment are faster. |
| **`sam build` or `deploy.sh` fails straight away** | Docker isn't running, the SAM CLI isn't installed, or credentials aren't set. On Windows, run the scripts from WSL or Git Bash. |
| **ECS: "Need at least 2 public subnets"** | Your default VPC's subnets don't route to an internet gateway. New accounts are fine; otherwise check the route tables. |
| **ECS URL doesn't load** | Wait 2–3 minutes for health checks. Check `aws logs tail /ecs/scheduling-agent`. If your public IP has changed since you deployed, the load balancer blocks you: its rule is only written on the first deploy, so run `./cleanup.sh` and deploy again, or add your new IP to the load balancer's security group. |
| **AgentCore status `CREATE_FAILED`** | Check the runtime's CloudWatch logs and that you may create IAM roles, ECR repositories and CodeBuild projects. |
| **Session ID rejected** | Runtime session IDs must be at least 33 characters; use a UUID, or two in a row. |
| **Multi-agent: `401` / `403` after an hour** | The Cognito token expired. Run `cognito_refresh_token`; the deployed agents also hold a copy (`AGENT_BEARER_TOKEN`), so redeploy them with the fresh token. |
| **Gateway target creation fails right after creating the role** | IAM changes take a few seconds to propagate. Wait and rerun the cell. |

| What you see | What it usually means and how to fix it |
|---|---|
| **"Address already in use" running the multi-agent lab locally** | Something else is on `8000`, `9000` or `9001`. Stop it, or stop old copies of the servers. |

## Clean-up checklist

| Lab | Clean up with | What's left behind if you forget |
|---|---|---|
| **Lambda** | `./cleanup.sh` | The function, API and image; idle Lambda costs little, but anyone who finds the URL can run up your Bedrock bill |
| **ECS Fargate** | `./cleanup.sh` | The running task and the load balancer, both billing every hour |
| **AgentCore Runtime** | Steps 8–9 of the notebook ( `stop_runtime_session` , `runtime.destroy` ) | The runtime and endpoint, open sessions until they time out, the image in ECR |
| **AgentCore Gateway** | Step 7 of the notebook | The gateway, credential provider, IAM role and S3 bucket |
| **Multi-agent** | `agentcore destroy` ×3 and `delete-user-pool` | Three runtimes, ECR images and the Cognito pool; log groups are kept until you delete them |

## Self-check

Ten quick questions. Pick an answer to see why it's right or wrong.

**1. In the Lambda lab, why is the model client created at module level but a new Agent created inside the handler?**

Agents can't be created at module level

The model client is expensive and holds no conversation; the agent holds history, and a warm environment serves many different callers

It makes cold starts disappear

Lambda requires it

**2. The Lambda lab sits behind an API Gateway REST API. Roughly how long can a request take before it times out by default?**

29 seconds

15 minutes

60 minutes

8 hours

**3. Your Lambda agent uses global.anthropic.claude-sonnet-4-6 and gets AccessDeniedException. What's most likely missing from the IAM policy?**

s3:GetObject

ecs:RunTask

Nothing; it must be a model ID typo

The inference-profile ARN (and the region-less foundation-model ARN)

**4. On ECS, what keeps two copies of your agent running and replaces one that crashes at 3 AM?**

The task definition

The load balancer

The ECS service

CloudWatch

**5. In the ECS lab, how does the agent know "the 9 AM slot" refers to Dr. Arti's knee replacement?**

The browser sends the full history with each message and the server seeds a fresh agent with it

The container remembers every user

The load balancer keeps sessions

AgentCore Memory

**6. A user takes a minute to reply between messages in an AgentCore Runtime session. What happens to CPU billing during that minute?**

You pay full CPU for the whole minute

The session is terminated

CPU isn't billed while the agent is just waiting

The microVM is shared with another user meanwhile

**7. What is the default idle timeout for an AgentCore Runtime session?**

1 minute

15 minutes

60 minutes

8 hours

**8. What do the four BedrockAgentCoreApp lines add to a Strands agent?**

A Dockerfile

A Cognito user pool

An MCP server

An HTTP server on port 8080 with POST /invocations and GET /ping

**9. Your gateway exposes 200 tools. How does the agent avoid loading all 200 definitions into every prompt?**

Use the gateway's semantic search tool, x_amz_bedrock_agentcore_search

Split the agent into 200 agents

Use a bigger model

Delete 190 tools

**10. The agent must read a user's own Google Calendar on their behalf. Which flow does AgentCore Identity use?**

OAuth client credentials (machine to machine)

A shared admin API key in an environment variable

OAuth authorisation code grant: the user clicks Allow once and Identity stores and refreshes the tokens

No authentication

Score: 0 of 10 answered

→ **Up next: Section 7, evals, observability and governance**

Deployed isn't the same as production-ready. Next you'll find out how well your agent is doing (AgentCore Evaluations with a custom LLM-as-judge), what it's doing (tracing with OpenTelemetry into Jaeger, Langfuse, LangSmith and CloudWatch), and what it's allowed to do (Bedrock Guardrails for what it says, AgentCore Policy with Cedar for what it does). Folder: `section-07-evals-observability-governance`.

---

# A–Z Glossary

Every underlined word on this page, in one place.

### AgentCore Gateway

An AgentCore service that turns OpenAPI specs, Lambda functions, Smithy models and MCP servers into one catalogue of MCP tools, with authentication handled for you.

### AgentCore starter toolkit

A Python package (bedrock-agentcore-starter-toolkit) with a Runtime class and an agentcore CLI that configure, deploy, invoke and destroy AgentCore agents.

### Amazon API Gateway

A managed front door for HTTP APIs. It receives web requests and forwards them to a backend such as a Lambda function.

### Amazon Bedrock AgentCore

A set of AWS managed services built specifically for running agents in production: Runtime, Gateway, Identity, Policy, Memory, Observability, Evaluations, Browser and Code Interpreter.

### Amazon Cognito

AWS's user directory and sign-in service. It issues OAuth/OIDC tokens (JWTs) that other services can verify.

### Amazon DynamoDB

AWS's fully managed key-value and document database. Fast lookups by key, no servers to manage; a natural home for session state.

### Amazon ECR

Elastic Container Registry: AWS's private store for container images, from which Lambda, ECS and AgentCore pull them.

### Amazon ECS

Elastic Container Service: AWS's service for running and keeping containers alive, behind load balancers, at the scale you choose.

### Amazon EventBridge

AWS's event bus: rules match events (like a file landing in S3) and route them to targets; EventBridge Scheduler also runs things on a timetable.

### Amazon RDS Proxy

A managed proxy in front of a relational database that pools and shares connections, so many short-lived clients don't exhaust the database.

### Amazon Redshift and Athena

Two AWS analytics services: Redshift is a data warehouse; Athena runs SQL directly over files in S3. Both keep heavy reports off your live database.

### Amazon SageMaker

AWS's platform for building, training and hosting machine-learning models. A SageMaker endpoint serves a trained model's predictions.

### Amazon SQS

Simple Queue Service: a managed message queue. Producers drop messages in, consumers take them out at their own pace, smoothing out bursts.

### Application Load Balancer

A managed load balancer that receives web traffic and spreads it across healthy targets, such as your containers.

### ARM64 (Graviton)

A processor architecture used by AWS Graviton chips and Apple Silicon. Container images must be built for the architecture they'll run on.

### ARN (Amazon Resource Name)

AWS's unique address for a resource, such as arn:aws:bedrock:us-east-1::foundation-model/.... IAM policies use ARNs to say what may be accessed.

### AWS CloudFormation

AWS's infrastructure-as-code service. A template describes resources; CloudFormation creates them together as a stack and can delete them together.

### AWS CodeBuild

AWS's managed build service. AgentCore's toolkit uses it to build your container image in the cloud, so you don't need Docker locally.

### AWS Fargate

A way to run containers on ECS without managing servers: you choose CPU and memory per task, AWS provides and patches the machines.

### AWS KMS

Key Management Service: AWS's managed service for creating and controlling the encryption keys that protect your data.

### AWS Lambda

AWS's serverless functions: you upload code with a handler, and AWS runs it when an event arrives, scaling from zero to many copies automatically.

### AWS SAM

The Serverless Application Model: a short YAML template format plus a CLI (sam build, sam deploy) for defining and deploying serverless apps.

### AWS Step Functions

A service for wiring AWS steps into visual workflows (do this, then that, retry on failure), often used to orchestrate other services.

### Cedar

An open-source policy language created by AWS for writing precise allow/deny rules. AgentCore Policy uses it to check every tool call before it runs.

### CIDR notation

A compact way to write a range of IP addresses: 203.0.113.7/32 means exactly one address; 0.0.0.0/0 means every address on the internet.

### Circuit breaker

A safety switch around calls to another service: after repeated failures it stops calling for a while and fails fast, then tries again later, so one broken service doesn't stall everything.

### Cold start

The extra delay when a new execution environment has to start and run your setup code before it can handle a request.

### Container

A sealed package of your code plus everything it needs to run (Python, libraries, settings), built from a Dockerfile. It runs the same on a laptop and in the cloud.

### Dead-letter queue

A side queue where messages go after failing several times, so they can be inspected and replayed instead of being lost or retried forever.

### Distributed tracing

Recording one request's journey across many services as a single linked timeline, so you can see where time went and where it failed.

### ECS service

The part of ECS that keeps a chosen number of copies (tasks) of your container running, replaces failed ones and rolls out new versions.

### EHR (Electronic Health Record)

The hospital system holding patients' medical histories, diagnoses, procedures and treatments.

### Embedding

A list of numbers representing the meaning of a piece of text. Texts with similar meanings have embeddings that are close together.

### Execution environment

The small, isolated runtime Lambda creates to run your function. It's reused for many invocations while traffic flows, then recycled when idle.

### Exponential backoff

Retrying after waits that double each time (1 s, 2 s, 4 s…), usually with a little randomness, so retries don't overwhelm a struggling service.

### FastAPI

A popular Python framework for building web APIs. You decorate functions with routes like @app.post("/schedule") and it handles the HTTP.

### Gateway target

One API registered with an AgentCore Gateway. The gateway generates one MCP tool for each of its operations.

### Graceful degradation

Keeping a reduced but useful service when something fails, for example answering with a preliminary assessment instead of crashing.

### Health check

A regular probe (here, a request to /health or /ping) that tells the platform whether a container is working. Failing ones are taken out of service and replaced.

### IAM (Identity and Access Management)

AWS's permission system. Roles and policies say exactly which AWS actions a person or a piece of software may perform on which resources.

### Inference profile

A Bedrock model ID such as global.anthropic.claude-sonnet-4-6 that can route requests across regions for capacity. IAM must allow the profile and the models behind it.

### Internet gateway

The component that connects a VPC to the public internet. Subnets whose routes point to it are public.

### JWT (JSON Web Token)

A compact, signed token that carries who someone is and what they may do. Services verify the signature instead of asking for a password.

### Lambda durable functions

A Lambda feature for long workflows: progress is checkpointed, and when the function reruns it skips completed steps and resumes where it left off.

### Least privilege

Giving each person or program only the permissions it truly needs, and nothing more, so a mistake or breach can do less damage.

### Listener

The part of a load balancer that waits on a port (here, 80) and decides where incoming requests go.

### LLM-as-judge

Using a language model to score another model's or agent's output against criteria such as accuracy, faithfulness or helpfulness.

### microVM

A very lightweight virtual machine that starts in a fraction of a second but keeps the strong isolation of a full VM: its own kernel, memory and files.

### OIDC (OpenID Connect)

A standard identity layer on top of OAuth 2.0 that lets a service verify who a user is from a token issued by an identity provider.

### OpenAPI

A standard JSON or YAML document describing a REST API: its endpoints, parameters and responses. Also known by its older name, Swagger.

### OpenTelemetry

An open standard for collecting traces, metrics and logs from software, supported by most monitoring tools.

### Provisioned concurrency

A Lambda setting that keeps a fixed number of environments warm and ready, removing cold starts but costing money even when idle.

### Rate limit

A cap on how many requests a client may make in a period. Over the limit, APIs answer 429 Too Many Requests, often with a Retry-After header.

### Semantic search

Finding things by meaning rather than exact words, by comparing embeddings. Gateway uses it to pick the few relevant tools out of hundreds.

### Serverless

Running code without managing servers: the cloud provider starts, scales and stops the compute for you, and you pay only while it runs.

### Smithy

AWS's open-source language for describing service APIs, used to define many AWS services; Gateway can turn Smithy models into tools.

### Subnet

A slice of a VPC's address range in one availability zone. Public subnets route to the internet; private subnets don't.

### Target group

The set of destinations (here, your containers) a load balancer forwards requests to, with a health check that removes unhealthy ones.

### Task definition

ECS's recipe for a container: which image, how much CPU and memory, which ports, environment variables, roles and where logs go.

### Token vault

AgentCore Identity's encrypted store for OAuth tokens and API keys, bound to a specific agent and user, with every access logged.

### uvicorn

A fast Python web server that runs FastAPI apps and listens for HTTP requests on a port.

### VPC (Virtual Private Cloud)

Your own private network inside AWS, with its own address range, subnets, routing and firewalls.

© 2026 Shivank Agrawal, Future with Shivank. Agentic AI on AWS, Section 6. Course code is provided under the MIT License.

AWS service names, limits, prices and library versions change often. When something here differs from what you see, the current AWS documentation wins.